Machine Learning - Final Project

Submitted by: Eli Titiyevsky 314654310 and Daphne Bistolas 209071638

```
!pip install numpy matplotlib pandas seaborn tensorflow tensorflow-
datasets scikit-learn
```

```
Requirement already satisfied: numpy in d:\anaconda\lib\site-packages
(1.26.4)
Requirement already satisfied: matplotlib in d:\anaconda\lib\site-
packages (3.8.4)
Requirement already satisfied: pandas in d:\anaconda\lib\site-packages
(2.2.2)
Requirement already satisfied: seaborn in d:\anaconda\lib\site-
packages (0.13.2)
Requirement already satisfied: tensorflow in d:\anaconda\lib\site-
packages (2.20.0)
Requirement already satisfied: tensorflow-datasets in d:\anaconda\lib\
site-packages (4.9.9)
Requirement already satisfied: scikit-learn in d:\anaconda\lib\site-
packages (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in d:\anaconda\lib\
site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in d:\anaconda\lib\site-
packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in d:\anaconda\lib\
site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in d:\anaconda\lib\
site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in d:\anaconda\lib\
site-packages (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in d:\anaconda\lib\site-
packages (from matplotlib) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in d:\anaconda\lib\
site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in d:\anaconda\
lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in d:\anaconda\lib\site-
packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in d:\anaconda\lib\site-
packages (from pandas) (2023.3)
Requirement already satisfied: absl-py>=1.0.0 in d:\anaconda\lib\site-
packages (from tensorflow) (2.3.1)
Requirement already satisfied: astunparse>=1.6.0 in d:\anaconda\lib\
site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in d:\anaconda\
lib\site-packages (from tensorflow) (25.9.23)
```

```
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
d:\anaconda\lib\site-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google_pasta>=0.1.1 in d:\anaconda\lib\
site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in d:\anaconda\lib\
site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in d:\anaconda\lib\
site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: protobuf>=5.28.0 in d:\anaconda\lib\
site-packages (from tensorflow) (6.33.0)
Requirement already satisfied: requests<3,>=2.21.0 in d:\anaconda\lib\
site-packages (from tensorflow) (2.32.2)
Requirement already satisfied: setuptools in d:\anaconda\lib\site-
packages (from tensorflow) (69.5.1)
Requirement already satisfied: six>=1.12.0 in d:\anaconda\lib\site-
packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in d:\anaconda\lib\
site-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing_extensions>=3.6.6 in d:\
anaconda\lib\site-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in d:\anaconda\lib\site-
packages (from tensorflow) (1.14.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in d:\anaconda\lib\
site-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.20.0 in d:\anaconda\lib\
site-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in d:\anaconda\lib\site-
packages (from tensorflow) (3.12.0)
Requirement already satisfied: h5py>=3.11.0 in d:\anaconda\lib\site-
packages (from tensorflow) (3.11.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in d:\anaconda\
lib\site-packages (from tensorflow) (0.5.3)
Requirement already satisfied: dm-tree in d:\anaconda\lib\site-
packages (from tensorflow-datasets) (0.1.9)
Requirement already satisfied: etils>=1.9.1 in d:\anaconda\lib\site-
packages (from etils[edc,enp,epath,epy,etree]>=1.9.1; python_version
>= "3.11"->tensorflow-datasets) (1.13.0)
Requirement already satisfied: immutabledict in d:\anaconda\lib\site-
packages (from tensorflow-datasets) (4.2.2)
Requirement already satisfied: promise in d:\anaconda\lib\site-
packages (from tensorflow-datasets) (2.3)
Requirement already satisfied: psutil in d:\anaconda\lib\site-packages
(from tensorflow-datasets) (5.9.0)
Requirement already satisfied: pyarrow in d:\anaconda\lib\site-
packages (from tensorflow-datasets) (14.0.2)
Requirement already satisfied: simple_parsing in d:\anaconda\lib\site-
packages (from tensorflow-datasets) (0.1.7)
Requirement already satisfied: tensorflow-metadata in d:\anaconda\lib\
site-packages (from tensorflow-datasets) (1.17.2)
```

```
Requirement already satisfied: toml in d:\anaconda\lib\site-packages
(from tensorflow-datasets) (0.10.2)
Requirement already satisfied: tqdm in d:\anaconda\lib\site-packages
(from tensorflow-datasets) (4.66.4)
Requirement already satisfied: scipy>=1.6.0 in d:\anaconda\lib\site-
packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in d:\anaconda\lib\site-
packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in d:\anaconda\
lib\site-packages (from scikit-learn) (2.2.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in d:\anaconda\lib\
site-packages (from astunparse>=1.6.0->tensorflow) (0.43.0)
Requirement already satisfied: einops in d:\anaconda\lib\site-packages
(from etils[edc,enp,epath,epy,etree]>=1.9.1; python_version >= "3.11"-
>tensorflow-datasets) (0.8.1)
Requirement already satisfied: fsspec in d:\anaconda\lib\site-packages
(from etils[edc,enp,epath,epy,etree]>=1.9.1; python_version >= "3.11"-
>tensorflow-datasets) (2024.3.1)
Requirement already satisfied: importlib_resources in d:\anaconda\lib\
site-packages (from etils[edc,enp,epath,epy,etree]>=1.9.1;
python_version >= "3.11"->tensorflow-datasets) (6.5.2)
Requirement already satisfied: zipp in d:\anaconda\lib\site-packages
(from etils[edc,enp,epath,epy,etree]>=1.9.1; python_version >= "3.11"-
>tensorflow-datasets) (3.17.0)
Requirement already satisfied: rich in d:\anaconda\lib\site-packages
(from keras>=3.10.0->tensorflow) (13.3.5)
Requirement already satisfied: namex in d:\anaconda\lib\site-packages
(from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in d:\anaconda\lib\site-packages
(from keras>=3.10.0->tensorflow) (0.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in d:\
anaconda\lib\site-packages (from requests<3,>=2.21.0->tensorflow)
(2.0.4)
Requirement already satisfied: idna<4,>=2.5 in d:\anaconda\lib\site-
packages (from requests<3,>=2.21.0->tensorflow) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in d:\anaconda\lib\
site-packages (from requests<3,>=2.21.0->tensorflow) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in d:\anaconda\lib\
site-packages (from requests<3,>=2.21.0->tensorflow) (2025.10.5)
Requirement already satisfied: markdown>=2.6.8 in d:\anaconda\lib\
site-packages (from tensorboard~=2.20.0->tensorflow) (3.4.1)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in d:\anaconda\lib\site-packages (from tensorboard~=2.20.0-
>tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in d:\anaconda\lib\
site-packages (from tensorboard~=2.20.0->tensorflow) (3.0.3)
Requirement already satisfied: attrs>=18.2.0 in d:\anaconda\lib\site-
packages (from dm-tree->tensorflow-datasets) (23.1.0)
Requirement already satisfied: docstring-parser<1.0,>=0.15 in d:\
```

```
anaconda\lib\site-packages (from simple_parsing->tensorflow-datasets)
(0.17.0)
Requirement already satisfied: googleapis-common-protos<2,>=1.56.4 in
d:\anaconda\lib\site-packages (from tensorflow-metadata->tensorflow-
datasets) (1.71.0)
Requirement already satisfied: colorama in d:\anaconda\lib\site-
packages (from tqdm->tensorflow-datasets) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in d:\anaconda\lib\
site-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow)
(2.1.3)
Requirement already satisfied: markdown-it-py<3.0.0,>=2.2.0 in d:\
anaconda\lib\site-packages (from rich->keras>=3.10.0->tensorflow)
(2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in d:\anaconda\
lib\site-packages (from rich->keras>=3.10.0->tensorflow) (2.15.1)
Requirement already satisfied: mdurl~=0.1 in d:\anaconda\lib\site-
packages (from markdown-it-py<3.0.0,>=2.2.0->rich->keras>=3.10.0-
>tensorflow) (0.1.0)
```

Neccesery Libraries Loadup:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import pandas as pd
import os

import seaborn as sns
from seaborn import heatmap

import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Input, Dense, Dropout,
GlobalAveragePooling2D
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.utils import to_categorical



import sklearn
from sklearn import decomposition
```

```
from sklearn.manifold import TSNE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix, precision_score, recall_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

The dataset will be loaded directly in TensorFlow format to facilitate model development and streamline preprocessing, instead of manually extracting the ZIP file and converting the data to a 3×3 configuration.

```
#Dataset loadup
X, Y =
tfds.as_numpy(tfds.load('colorectal_histology',split='train',batch_siz
e=-1, as_supervised=True,))

# Print the shape of the data and the unique labels of the data.
print('X.shape =',X.shape)
print('np.unique(Y) =',np.unique(Y))
```

```
WARNING:absl:Variant folder
/root/tensorflow_datasets/colorectal_histology/2.0.0 has no
dataset_info.json

Downloading and preparing dataset Unknown size (download: Unknown
size, generated: Unknown size, total: Unknown size) to
/root/tensorflow_datasets/colorectal_histology/2.0.0...

{"model_id":"0a9b330ff8104486be61f9f0945ecbd7","version_major":2,"vers
ion_minor":0}

{"model_id":"59868f853308485a9ef8dfa456c48e3f","version_major":2,"vers
ion_minor":0}

{"model_id":"c79f601f247444858170d943a0a8a1bf","version_major":2,"vers
ion_minor":0}

{"model_id":"17c7b8ed82824436b91f75fa979673e2","version_major":2,"vers
ion_minor":0}

{"model_id":"a1458c4e897442dfb3e289c540e3fe6d","version_major":2,"vers
ion_minor":0}

{"model_id":"97d57a63989e42f89f3b57f5f27d1ba0","version_major":2,"vers
ion_minor":0}

Dataset colorectal_histology downloaded and prepared to
/root/tensorflow_datasets/colorectal_histology/2.0.0. Subsequent calls
will reuse this data.
X.shape = (5000, 150, 150, 3)
np.unique(Y) = [0 1 2 3 4 5 6 7]
```

The division between training and testing data will be handled by the train_test_split function. We decided to allocate 80% of the data for training and 20% for testing. The assignment of each image to either the training or testing set will be performed randomly.

```
#Dataset split for training and testing.
x_train, x_test, y_train, y_test = train_test_split(X, Y,
test_size=0.2, random_state=42, stratify=Y)

# Print the shapes to verify the split
print(f"Shape of X_train (Training Features): {x_train.shape}")
print(f"Shape of X_test (Testing Features): {x_test.shape}")
print(f"Shape of y_train (Training Labels): {y_train.shape}")
print(f"Shape of y_test (Testing Labels): {y_test.shape}")
```

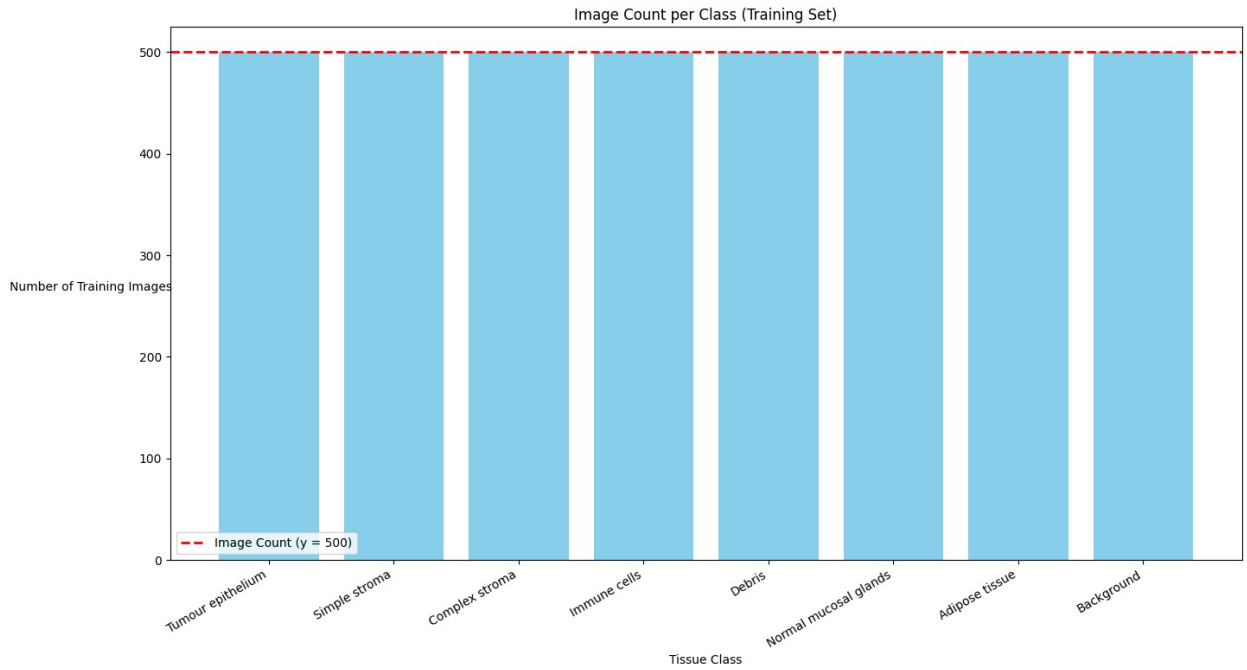
```
Shape of X_train (Training Features): (4000, 150, 150, 3)
Shape of X_test (Testing Features): (1000, 150, 150, 3)
Shape of y_train (Training Labels): (4000,)
Shape of y_test (Testing Labels): (1000,)
```

```
#Setting a more pleasent presentation of y labels.
class_names = ['Tumour epithelium', 'Simple stroma', 'Complex stroma',
'Immune cells', 'Debris', 'Normal mucosal glands', 'Adipose tissue',
'Background']
```

Bar Plot of Training Data:

The x-axis represents the different tissue types, and the y-axis indicates their frequency. From the graph, we can see that each tissue class contains 500 images in the training set.

```
#setting counts attibute for easier work.
counts = pd.Series(y_train).value_counts().sort_index()

#Barplot presentation.
plt.figure(figsize=(16,8))
plt.bar(class_names, counts.values, color='skyblue')
plt.axhline(counts.max(), color='red', linestyle='--', linewidth=2,
label='Image Count (y = 500)')
plt.title('Image Count per Class (Training Set)')
plt.xlabel('Tissue Class')
plt.ylabel('Number of Training Images', rotation=0, labelpad=40)
plt.xticks(rotation=30, ha='right')
plt.legend()
plt.show()
```



Checking Class Distribution in Training.

We will verify that each tissue class in the training set is evenly distributed during the training process.
This check is necessary to avoid both overfitting and underfitting.

```
if len(counts) > 1:
  balance_ratio = np.max(counts) / np.min(counts)
  print(f"Dataset balance ratio: {balance_ratio:.2f}")
  if balance_ratio > 2.0:
      print("Dataset is imbalanced! Class balancing may be required.")
  else:
      print("Dataset is relatively balanced.")
else:
      print("Not enough classes to assess balance.")
```

```
Dataset balance ratio: 1.00
Dataset is relatively balanced.
```

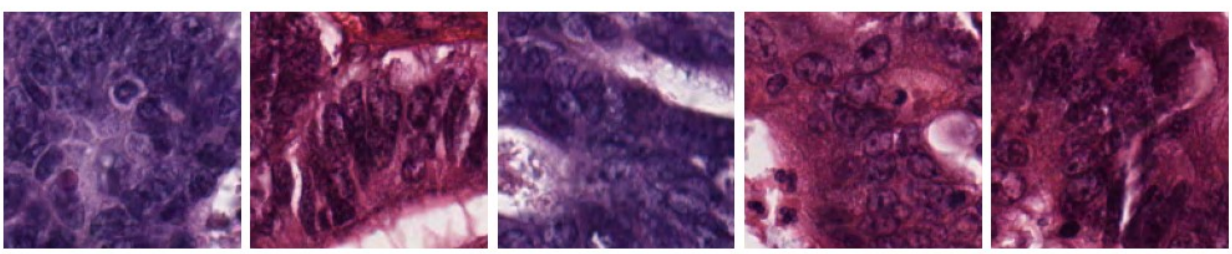
Plot of Example Images:

As required for the task, this section presents at least three example images from each tissue class.

```
n_per_class = 5
shuffle = False
```

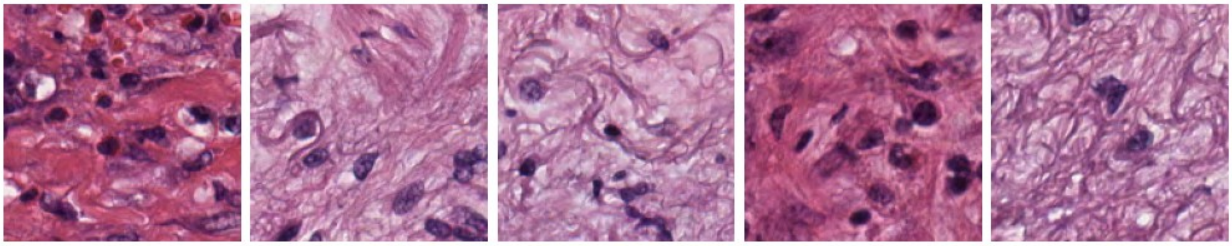
```python
for i in range(len(class_names)):
    idxs = np.where(Y == i)[0]
    if idxs.size == 0:
        continue
    if shuffle:
        np.random.shuffle(idxs)
    sample_indices = idxs[:n_per_class]

    fig, axes = plt.subplots(1, len(sample_indices), figsize=(10, 3))
    fig.suptitle(class_names[i], fontsize=16, y=0.95)

    if len(sample_indices) == 1:
        axes = [axes]

    for j, idx in enumerate(sample_indices):
        ax = axes[j]
        ax.imshow(X[idx])
        ax.axis('off')

    plt.tight_layout(pad=0.6, w_pad=0.6, h_pad=0.6)
    fig.subplots_adjust(top=0.85)
    plt.show()
```

Tumour epithelium

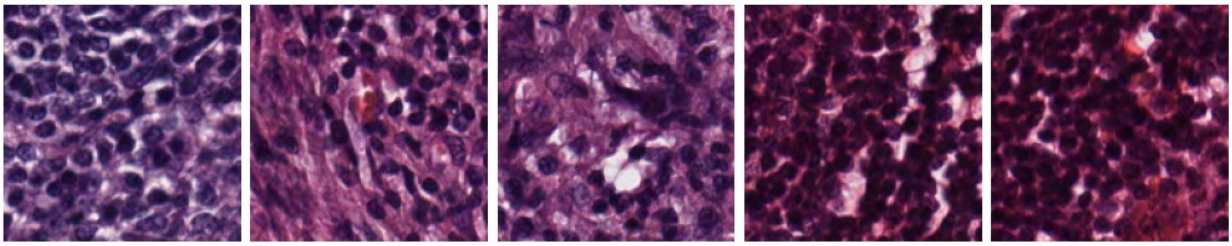


Simple stroma

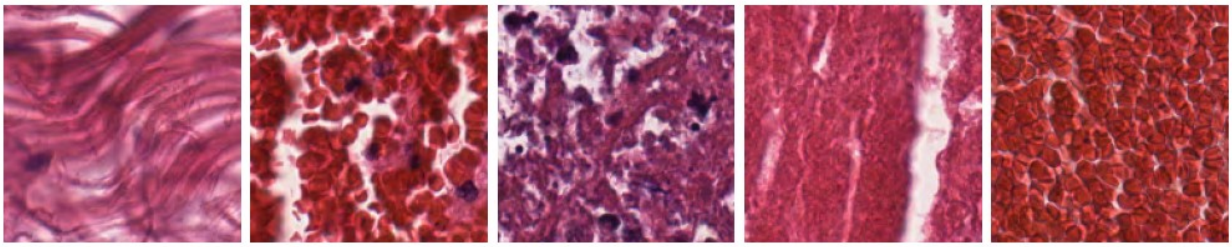
Complex stroma



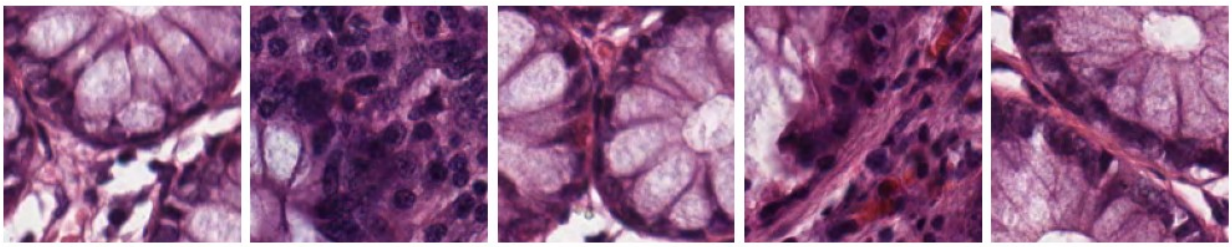
Immune cells



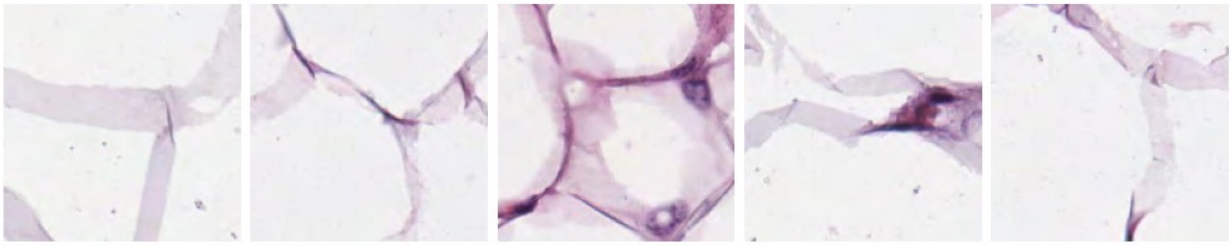
Debris



Normal mucosal glands



Adipose tissue



Background

Dimenssion Reduction Part:

As part of the task, we are required to reduce the dimenssions.

PCA Feature Processing:

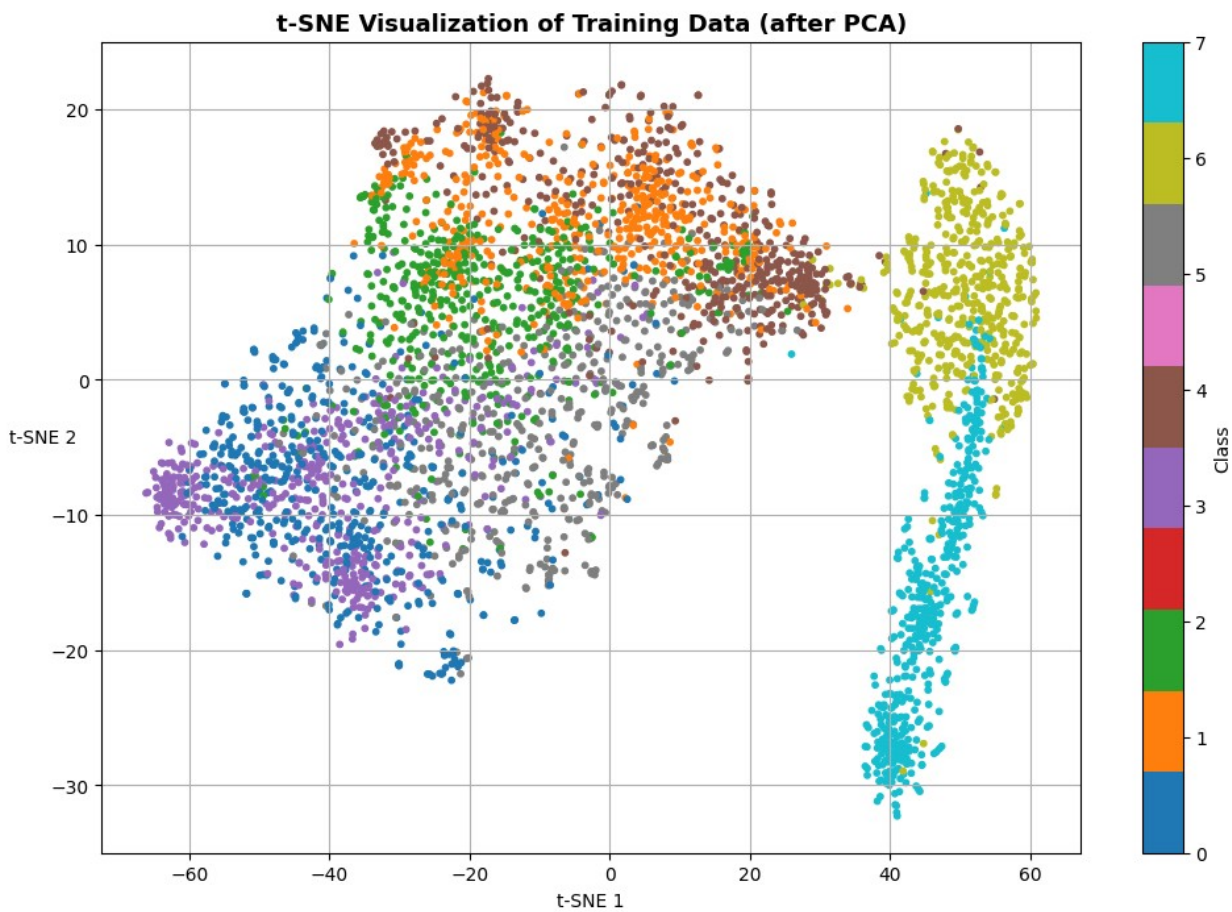
For processing the `x_train` images using PCA, the images must first be flattened from 2D into 1D.
We will use the `reshape` function to perform this transformation. Next, as required, we will apply PCA with 50 components. We will contitune the ranom_state value to 42 to follow the consisty of our project.

t-SNE setup:

We will initialize the t-SNE function using the PCA-transformed features as input.
The number of components will be set to 2 for visualization purposes.
The learning rate will be set to `'auto'`, and we will use the same `random_state` value to ensure reproducibility.

```
#Flattening function.
x_train_flatten = x_train.reshape(x_train.shape[0], -1) # Flattening the x_train data to 1D.

#Setting the PCA and runining the flatten x_train data.
pca_50 = decomposition.PCA(n_components=50, random_state=42) # Setting the PCA to 50 components.
x_pca_50 = pca_50.fit_transform(x_train_flatten) # Fit and transform the training data.
```

```
#Setting the t-SNE and running of previous PCA output.
tsne = TSNE(n_components = 2, learning_rate = "auto", init = 'pca',
perplexity=30, random_state=42)
Z = tsne.fit_transform(x_pca_50)

# t-SNE graph presentation. We can tell there is some seperation
between classes but significat overlap exists.
plt.figure(figsize = (12,8))
plt.scatter(Z[:,0], Z[:,1], s=10, c=y_train, cmap = "tab10")
plt.colorbar(label='Class')
plt.title('t-SNE Visualization of Training Data (after PCA)', fontsize
= 13, fontweight = 'bold')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2', rotation=0)
plt.grid(True)
plt.show()
```



Interpretation of t-SNE Visualization

The 2-D t-SNE plot displays how images from different tissue classes are distributed in feature space after dimensionality reduction.
We can observe that:

- Certain tissue types such as **Tumour epithelium** and **Normal mucosal glands** form relatively compact clusters, indicating strong intra-class similarity.
- Others, like **Complex stroma** or **Debris**, are more dispersed and partially overlap with neighboring clusters, suggesting visual similarities or noise.
- The overall separation implies that meaningful texture differences exist, yet some classes remain challenging to distinguish, justifying the need for advanced classifiers.

Feature Extraction Part:

In this part, we will use 3 learned feature extraction methods - Each method has their own pros and cons.

downsample and flatten method will resize images to `32x32` pixels and flatten it to `3072-D` vector.

PCA method will flatten the original images and apply PCA to reduce the dimenssion to `256 components`.

VGG-16 will use a pretrained model with some adjustements such as no `top layers` and output a `8192-D vector`.

```
# downsample method setup.
def downsample_flatten_features(data):
    downsampled_data = tf.image.resize(tf.convert_to_tensor(data),
[32, 32]).numpy()
    downsampled_data =
downsampled_data.reshape(downsampled_data.shape[0], -1)
    print(f"Downsample features shape: {downsampled_data.shape}")
    return downsampled_data

# PCA method setup.
def pca_features(data):
  data = data.reshape(data.shape[0], -1)
  pca = decomposition.PCA(n_components=256, random_state=42)
  pca_data = pca.fit_transform(data)
  print(f"PCA features shape: {pca_data.shape}")
  return pca_data

# VGG16 method setup.
def vgg_features(data):
  vgg_base = VGG16(weights='imagenet', include_top=False,
input_shape=(150, 150, 3))
  vgg_base.trainable = False
  data_prep = preprocess_input(data)
  vgg_data = vgg_base.predict(data_prep, batch_size=32, verbose=1)
  print(f"VGG-16 features shape: {vgg_data.shape}")
  return vgg_data.reshape(len(data), -1)
```

```
# Validation output for extracted features and their shape.
train_data_downsampled = downsample_flatten_features(x_train)
test_data_downsampled = downsample_flatten_features(x_test)
```

```
Downsample features shape: (4000, 3072)
Downsample features shape: (1000, 3072)
```

```
# Validation output for extracted features and their shape.
train_data_pca = pca_features(x_train)
test_data_pca = pca_features(x_test)
```

```
PCA features shape: (4000, 256)
PCA features shape: (1000, 256)
```

```
# Validation output for extracted features and their shape.
train_data_vgg = vgg_features(x_train)
test_data_vgg = vgg_features(x_test)
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 93s 744ms/step
VGG-16 features shape: (4000, 4, 4, 512)
32/32 ━━━━━━━━━━━━━━━━━━━━ 24s 733ms/step
VGG-16 features shape: (1000, 4, 4, 512)
```

Extracted Feture Display:

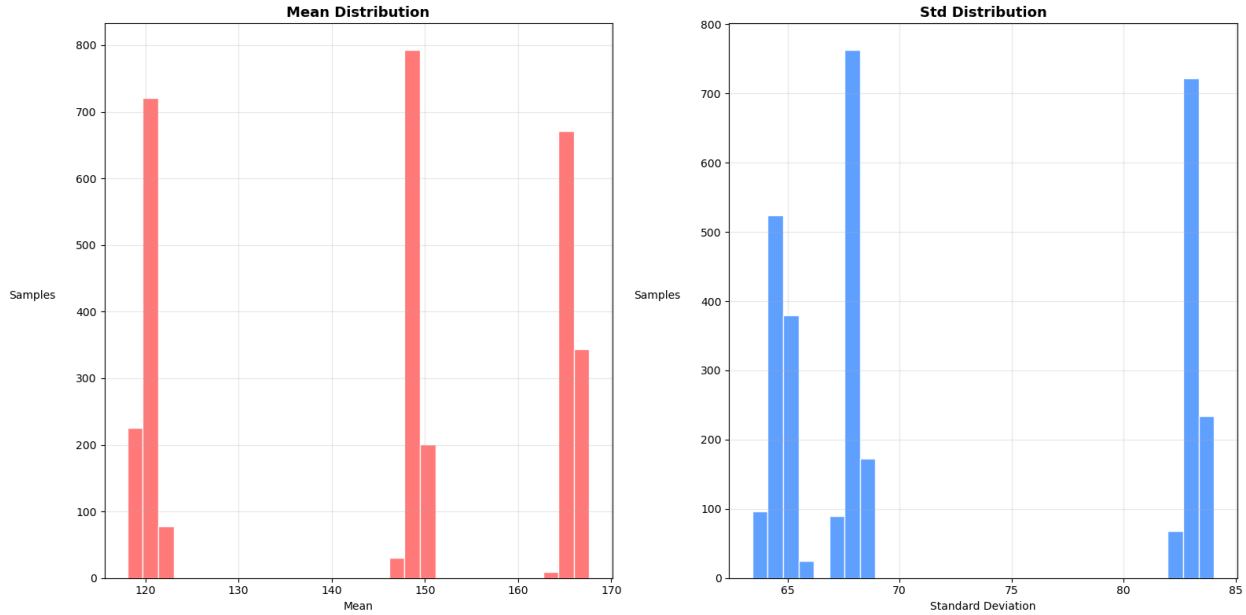
As instructed, We will presents the means and stds for each method and two histograms plots. One for *Mean* features and other for *STDS* features.

```
# Setting Histograds for means, stds and output for the mean and stds.
We will check the mean and stds to indicate us if normalization is
needed.
def plot_histograms(data):

  means = data.mean(axis=0)
  stds = data.std(axis=0)
  print(f"Mean: {means.mean()} Std: {stds.mean()}")
  print(f"Range of mean is: {means.min():.4f} to {means.max():.4f}")
  print(f"Range of stds is: {stds.min():.4f} to {stds.max():.4f}")

  if means.std() > 0.01 or stds.std() > 0.01:
    print("Normalization is required.")
  else:
    print("Normalization is not required.")

  plt.figure(figsize=(16, 8))

  plt.subplot(1, 2, 1)
  plt.hist(means, bins=30, color="#FF6B6B", edgecolor="white",
alpha=0.9)
```

```
  plt.title("Mean Distribution", fontsize=13, fontweight='bold')
  plt.xlabel("Mean")
  plt.ylabel("Samples", rotation=0, labelpad=40)
  plt.grid(alpha=0.3)

  plt.subplot(1, 2, 2)
  plt.hist(stds, bins=30, color="#4D96FF", edgecolor="white",
alpha=0.9)
  plt.title("Std Distribution", fontsize=13, fontweight='bold')
  plt.xlabel("Standard Deviation")
  plt.ylabel("Samples", rotation=0, labelpad=40)
  plt.grid(alpha=0.3)

  plt.tight_layout()
  plt.show()

# Normalization function via Scaler on the data.
def normalize(data):
  scaler = StandardScaler()
  normalized_data = scaler.fit_transform(data)
  return normalized_data

# t-SNE plot - Will be used only on normalized data.
def plot_tsne(normalized_data):
  tsne = TSNE(n_components=2, random_state=42, perplexity=30)
  tsne = tsne.fit_transform(normalized_data)
  plt.figure(figsize=(12, 8))
  plt.scatter(tsne[:, 0], tsne[:, 1], c = y_train, s=30, cmap="tab10")
  plt.colorbar(label = 'Class')
  plt.title("t-SNE Visualization of Extracted Data", fontsize = 13,
fontweight = 'bold')
  plt.xlabel("Dim 1")
  plt.ylabel("Dim 2" , rotation=0, labelpad=20)
  plt.grid()
  plt.tight_layout()
  plt.show()

def summary_table_of_experiments(data):
  summary = pd
  means = data.mean(axis=0)

# downsample and flatten extraction data presentation.
# The extracted features are smaller but are expected to be less
informative.

# Presenting the data via histogram plots.
plot_histograms(train_data_downsampled)
```

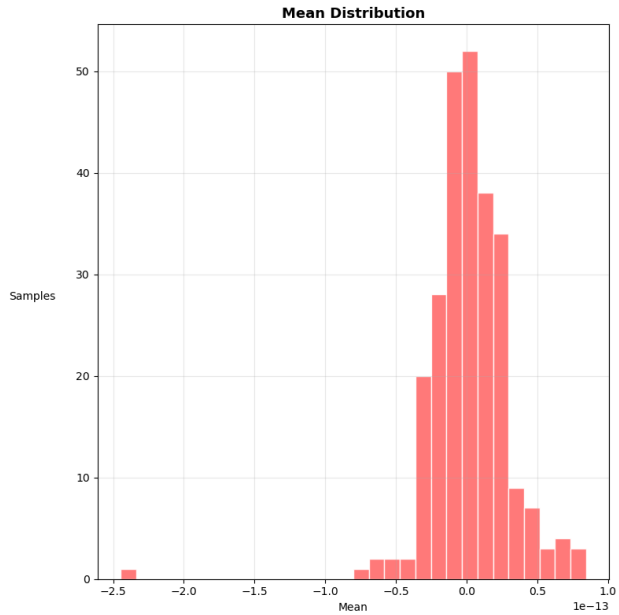
```
Mean: 144.99977111816406 Std: 71.91187286376953
Range of mean is: 118.0875 to 167.6277
```

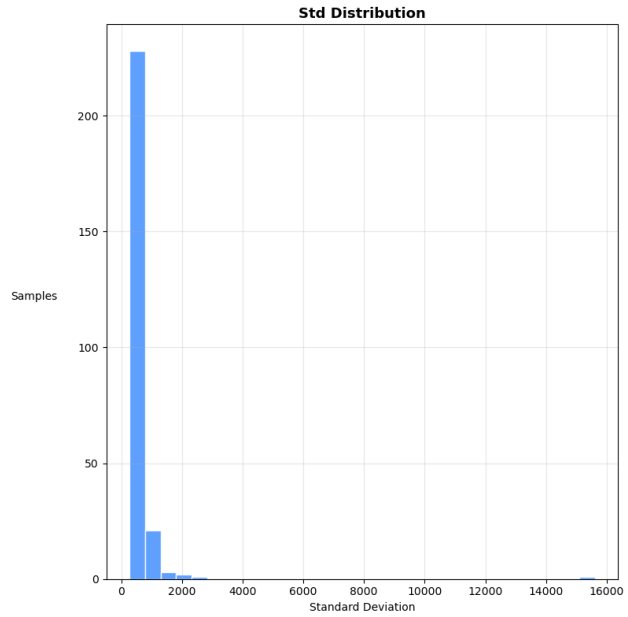
```
Range of stds is: 63.4288 to 84.0287
Normalization is required.
```




```
# PCA extraction data presentation.
# PCA successfully reduced dimensionality significantly while
retaining a substantial portion of the variance (about 95% explained
variance), making it a good candidate.
# Mean seems normalized but stds is need to be normalized.

# Presenting the data via histogram plots.
plot_histograms(train_data_pca)
```

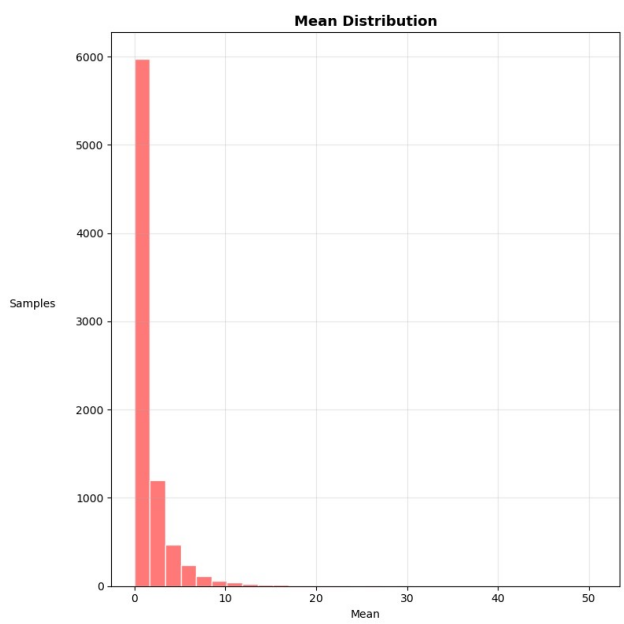
```
Mean: 2.1958872437402573e-15 Std: 555.1236272780906
Range of mean is: -0.0000 to 0.0000
Range of stds is: 272.3143 to 15606.2239
Normalization is required.
```

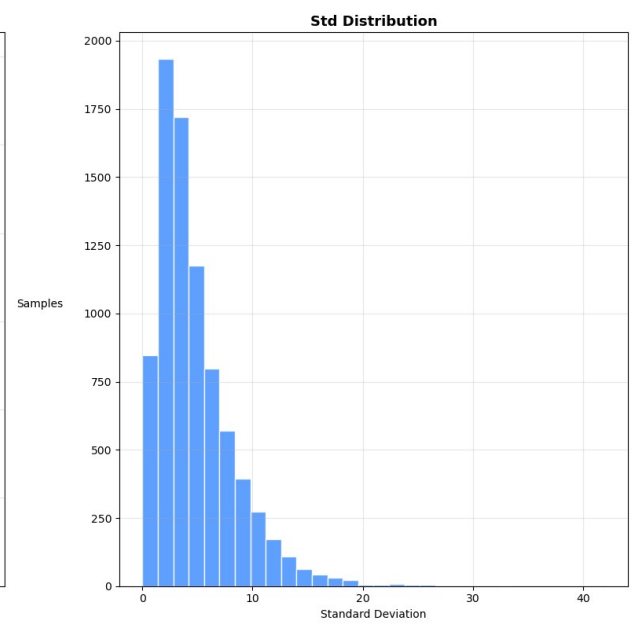





```
# VGG16 extraction data presentation.
# Feature extraction from VGG-16 produced higher-dimensional vectors
(8192), but these are expected to be semantically richer and have
better discriminative power.

# Presenting the data via histogram plots.
plot_histograms(train_data_vgg)
```

```
Mean: 1.5981287956237793 Std: 4.816383361816406
Range of mean is: 0.0000 to 50.8981
Range of stds is: 0.0000 to 41.9878
Normalization is required.
```





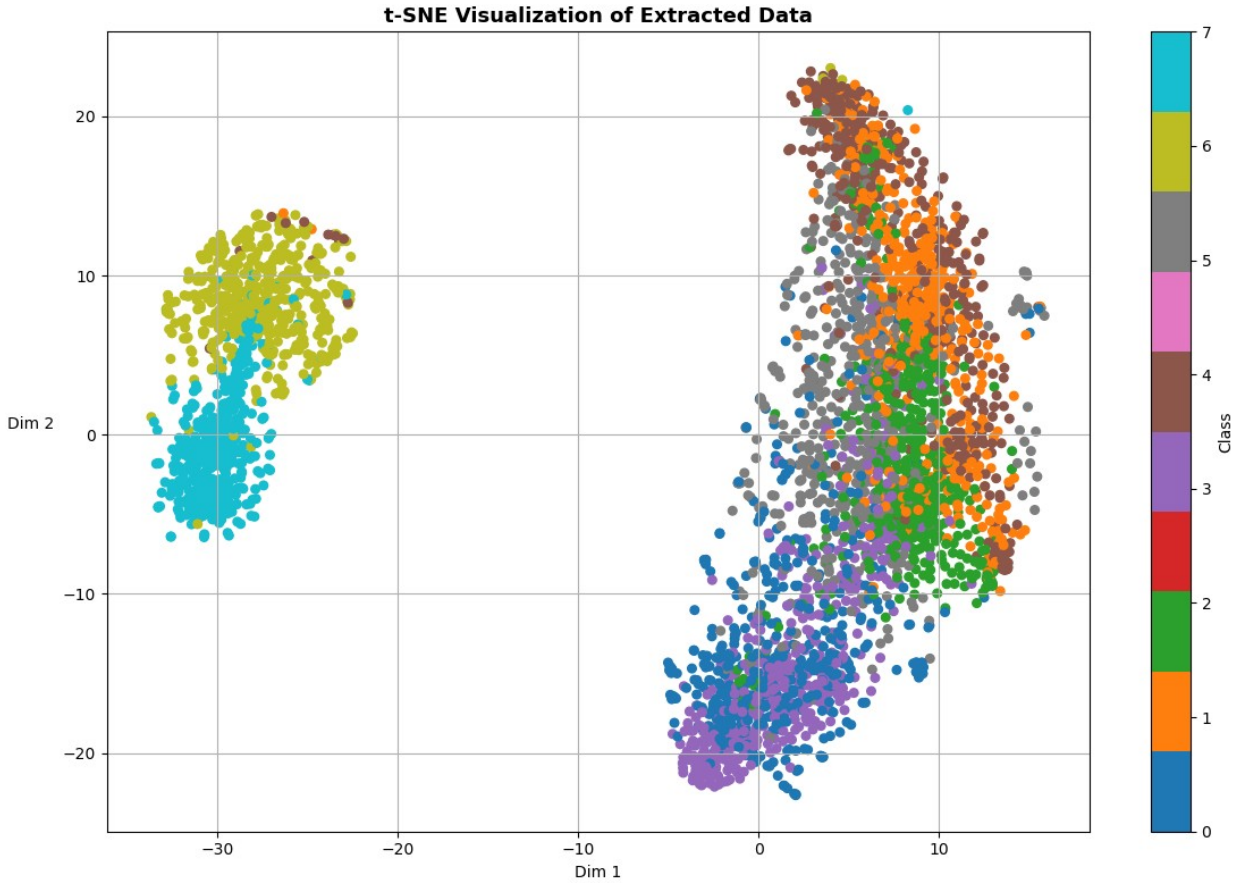
Why Normalization is Needed?

Feature values from different extraction methods (especially VGG-16 vs PCA vs Downsample) can exist on very different numeric scales.
Without normalization:

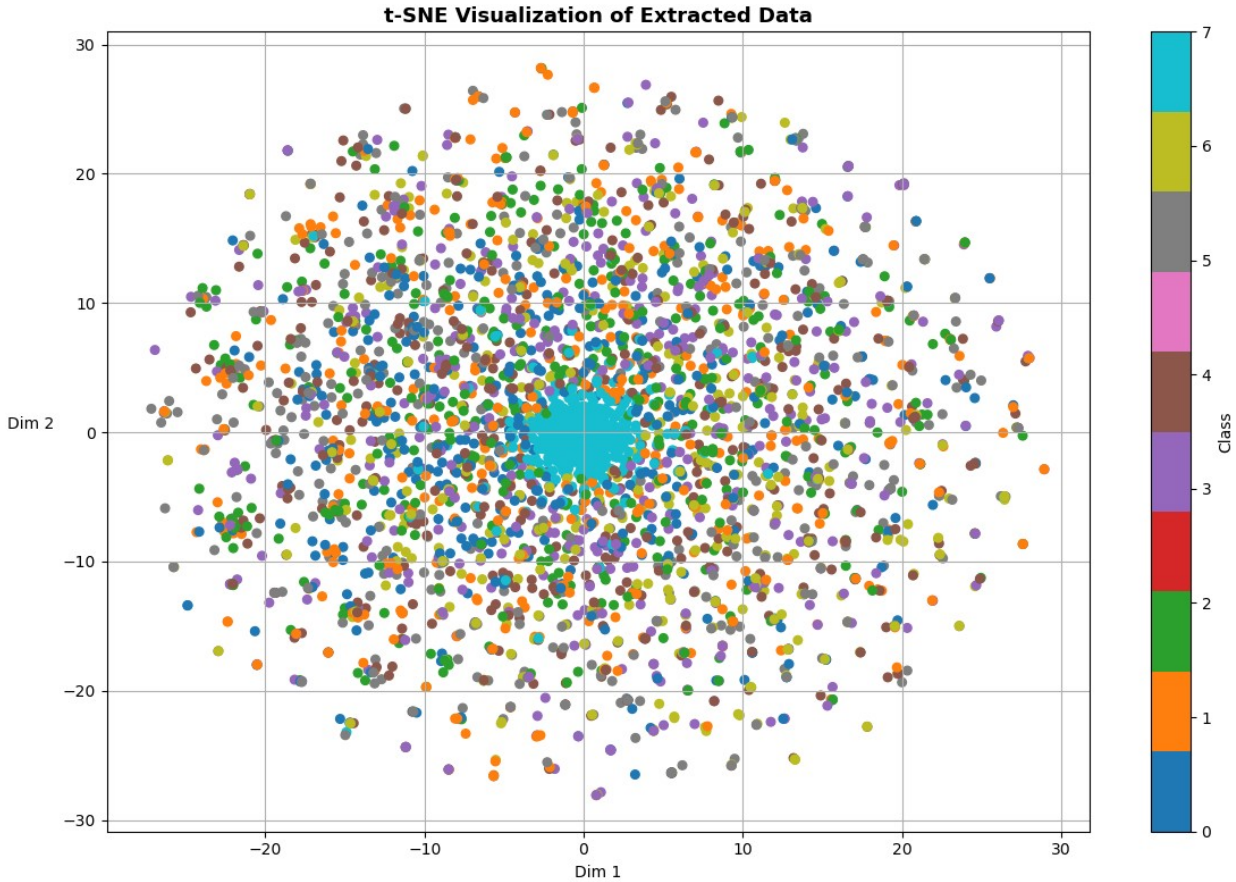
- Algorithms like **SVM** or **Neural Networks** may give higher importance to features with larger magnitudes.
- Optimization becomes unstable, and convergence is slower.

Therefore, all feature vectors are normalized (using `StandardScaler`) so that each feature has **zero mean and unit variance**.
This ensures fair contribution of all dimensions and improves both training stability and model accuracy.

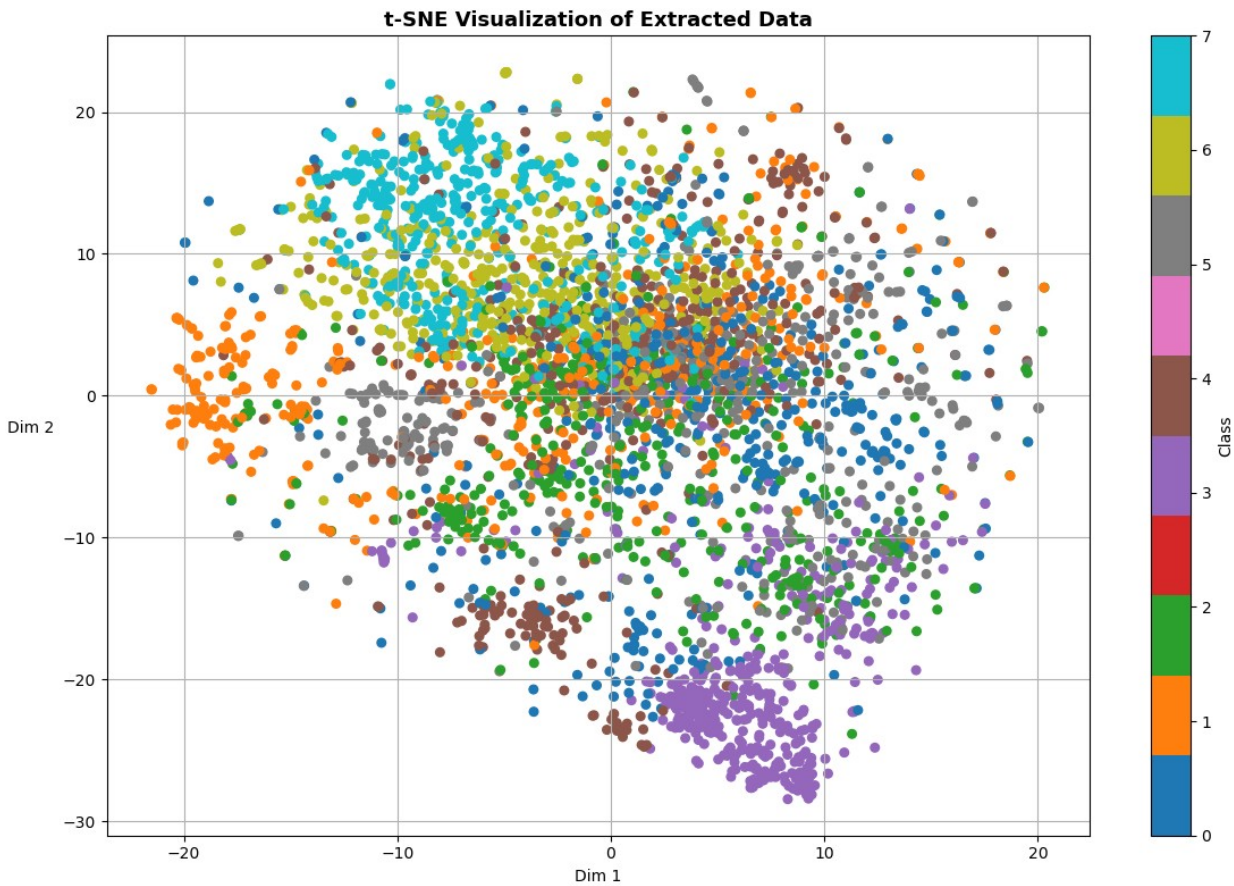
```
# Running normalization on data from downsampled method.
normalized_train_data_downsampled = normalize(train_data_downsampled)
normalized_test_data_downsampled = normalize(test_data_downsampled)

#Presenting the data in t-SNE plot.
plot_tsne(normalized_train_data_downsampled)

# As said before, Less imformative.
```




```
# Running normalization on data from PCA method.
normalized_train_data_pca = normalize(train_data_pca)
normalized_test_data_pca = normalize(test_data_pca)

#Presenting the data in t-SNE plot.
plot_tsne(normalized_train_data_pca)

# As said before, can be a good candidate.
```




```
# Running normalization on data from PCA method.
normalized_train_data_vgg = normalize(train_data_vgg)
normalized_test_data_vgg = normalize(test_data_vgg)

#Presenting the data in t-SNE plot.
plot_tsne(normalized_train_data_vgg)

# As said before, clusters appear more distinct than in other methods.
```



Basic Experiments

As instructed, we will now combine the three previously extracted feature sets with three different classification algorithms.
Each combination will be tested as a separate experiment.

In total, we will have **9 experiments** (3 feature extraction methods × 3 classifiers).
Additionally, we will include three extra tests using an **SVM** with a *linear kernel*, bringing the total to **12 experiments**.

Classifiers Overview

1. **Support Vector Machine (SVM):**
   Tested with both `rbf` and `linear` *kernels* to evaluate performance differences.
   althought **RBF** is more suited, We will check also the **linear** method for finding.

2. **Softmax Regression:**
   Implemented as **logistic regression** with `multi_class='multinomial'` and the **lbfgs** solver. we will set `max_iter = 1000` to ensure convergence.

3. **Neural Network (NN):**
   A simple `Sequential` model containing one `hidden layer` with `128` neurons

using the `ReLU` activation function,
followed by a `Dropout` layer (`rate = 0.2`)** and a `Softmax` output layer for multi-class classification.

Evaluation Metrics

Each experiment will be evaluated using the following performance metrics:

- **Accuracy**
- **F1-Score (Macro)**
- **Precision**
- **Recall**
- **Confusion Matrix**

In addition, For the neural network experiments, we will also display **training and validation curves** (Accuracy and Loss) to assess convergence and overfitting behavior.

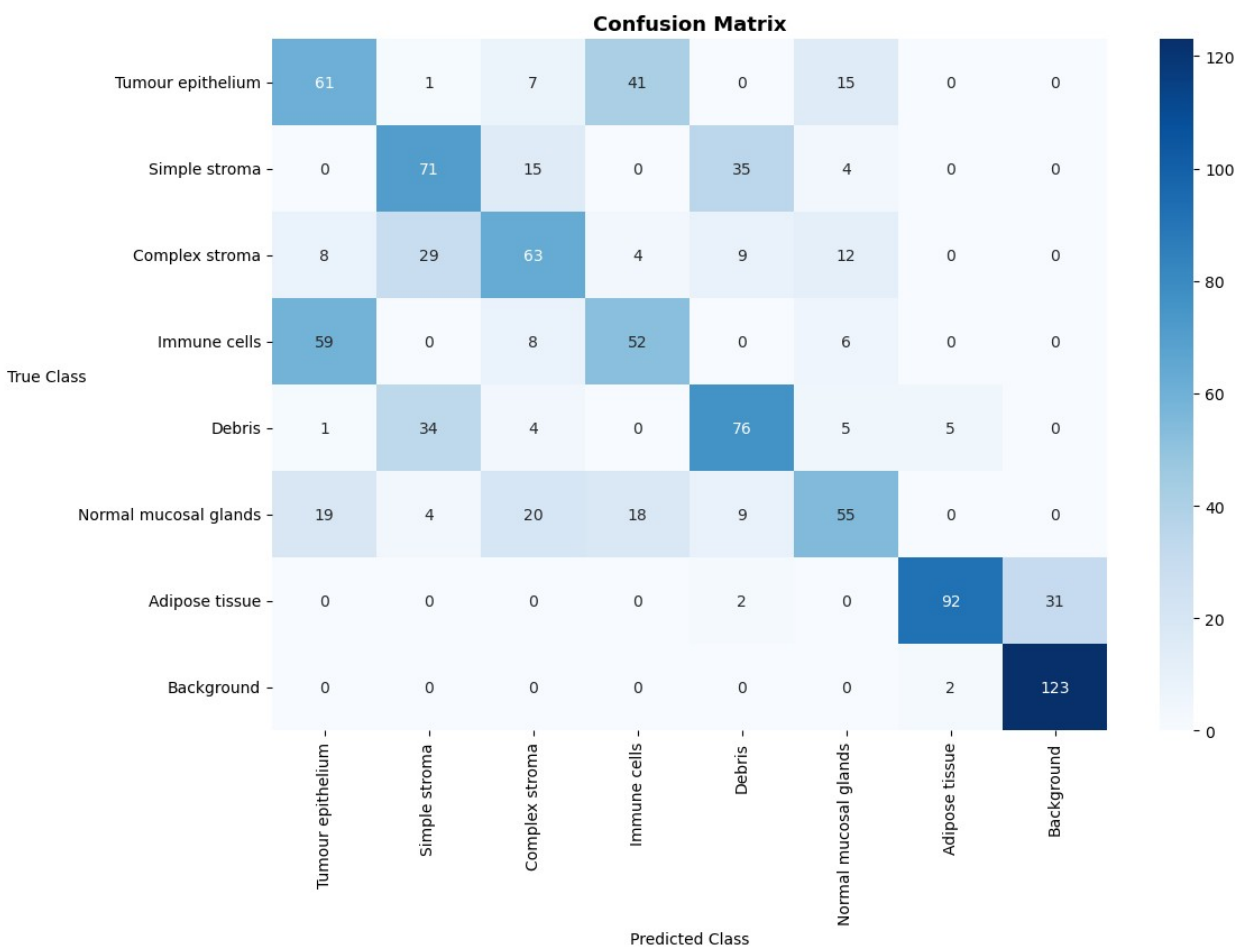
```
# Setting the SVM experiment
# SVM:
# kernel=kernel: Two different kernels will be used - 'rbf' as can
model non-linear data with complex, curved decision boundaries and
'linear' as it creates only straight-line boundaries, less righteous
for us - we will check for the out put.
# C=1.0: Regularization parameter. Smaller C values increase
regularization.
# gamma='scale': Kernel coefficient for RBF and linear which is a good
starting point.
# random_state = 42: as for consisesty along the project.
def svm_experiment(train_data, test_data, train_labels, test_labels,
kernel):
  model = sklearn.svm.SVC(kernel=kernel, C=1.0, gamma='scale',
random_state=42)
  model = model.fit(train_data, train_labels)
  y_pred_train = model.predict(train_data)
  y_pred_test = model.predict(test_data)
  return y_pred_train, y_pred_test

# Setting the softmax experiment.
# Softmax (Logistic Regression):
# multi_class='multinomial': Sets the model for multi-class
classification.
# solver='lbfgs': An optimizer that works well for small datasets.
# max_iter=1000: Maximum number of iterations for convergence.
Increased to 1000 to ensure convergence.
# random_state = 42: as for consisesty along the project.
def softmax_experiment(train_data, test_data, train_labels,
test_labels):
  model = LogisticRegression(multi_class='multinomial',
solver='lbfgs', max_iter=1000, random_state=42)
```

```
  model = model.fit(train_data, train_labels)
  y_pred_train = model.predict(train_data)
  y_pred_test = model.predict(test_data)
  return y_pred_train, y_pred_test

# Setting the NN experiment.
# Neural Network:
# Initializing a Sequential model.
# # Dense layer with 128 neurons and ReLU activation
# ReLU helps in training deep networks by preventing the vanishing
gradient problem.
# Dropout with a rate of 0.2 for regularization and preventing
overfitting
# Output layer with 8 neurons (one per class) and Softmax activation
# Softmax is used for multi-class classification, outputting
probabilities for each class.
def nn_experiment(train_data, test_data, train_labels, test_labels):
  model = Sequential([Dense(128, activation='relu',
input_shape=(train_data.shape[1],)), Dropout(0.2), Dense(8,
activation='softmax')])
  model.compile(optimizer=Adam(learning_rate=0.001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
  model_history = model.fit(train_data, train_labels, epochs=50,
batch_size=64, validation_data=(test_data, test_labels), verbose = 0)
  y_pred_train_probs = model.predict(train_data, verbose = 0)
  y_pred_test_probs = model.predict(test_data, verbose = 0)
  y_pred_train = np.argmax(y_pred_train_probs, axis=1)
  y_pred_test = np.argmax(y_pred_test_probs, axis=1)
  return y_pred_train, y_pred_test, model_history

# Setting an evalutation function for more suitabe use.
def evaluation(y_pred_train, y_pred_test, y_train, y_test):
  print("Evaluation Report:")
  print(f"Training Accuracy: {accuracy_score(y_train,
y_pred_train):.3f}")
  print(f"Testing Accuracy: {accuracy_score(y_test,
y_pred_test):.3f}")
  print(f"Training F1 Score: {f1_score(y_train, y_pred_train,
average='macro'):.3f}")
  print(f"Testing F1 Score: {f1_score(y_test, y_pred_test,
average='macro'):.3f}")
  print(f"Training Precision: {precision_score(y_train, y_pred_train,
average='weighted'):.3f}")
  print(f"Testing Precision: {precision_score(y_test, y_pred_test,
average='weighted'):.3f}")
  print(f"Training Recall: {recall_score(y_train, y_pred_train,
average='weighted'):.3f}")
  print(f"Testing Recall: {recall_score(y_test, y_pred_test,
average='weighted'):.3f}")
```

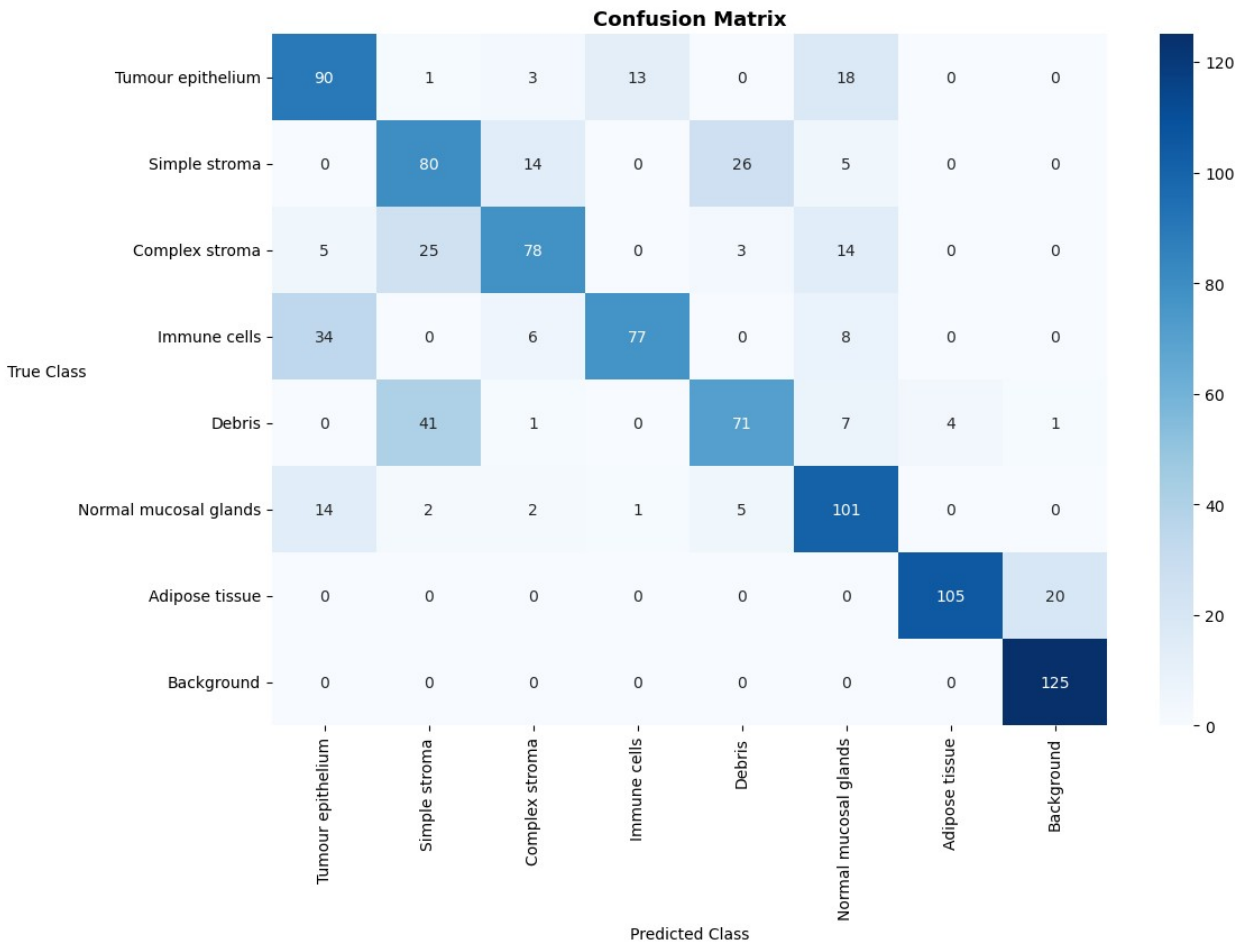
```
  overfit = accuracy_score(y_train, y_pred_train) -
accuracy_score(y_test, y_pred_test)
  print(f"Overfitting Check: {overfit:.3f}")
  if overfit <= 0:
    print("Excellent generalization - Test performance matches or
exceeds training.")
  elif overfit < 0.05:
    print("Insignificant overfitting.")
  elif overfit < 0.15:
    print("Slight overfitting detected.")
  else:
    print("Significant overfitting detected. Model may need more
regularization.")


  cm = confusion_matrix(y_test, y_pred_test)
  plt.figure(figsize=(12, 8))
  heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=class_names, yticklabels=class_names)
  plt.title('Confusion Matrix', fontsize=13, fontweight='bold')
  plt.xlabel('Predicted Class')
  plt.ylabel('True Class', rotation=0, labelpad=20)
  plt.show()

# Plot function for only NN experiemtns to show the training and loss
proccess.
def plot_vgg_history(model_history):
  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
  ax1.plot(model_history.history['accuracy'], label='Training
Accuracy')
  ax1.plot(model_history.history['val_accuracy'], label='Validation
Accuracy')
  ax1.set_xlabel('Epoch')
  ax1.set_ylabel('Accuracy')
  ax1.set_title('Training and Validation Accuracy')
  ax1.legend()
  ax1.grid(True)

  ax2.plot(model_history.history['loss'], label='Training Loss')
  ax2.plot(model_history.history['val_loss'], label='Validation Loss')
  ax2.set_xlabel('Epoch')
  ax2.set_ylabel('Loss')
  ax2.set_title('Training and Validation Loss')
  ax2.legend()
  ax2.grid(True)

  plt.tight_layout()
  plt.show()
```

```python
# Running the SVM with 'linear' kernel with downsampled data.
down_svm = svm_experiment(normalized_train_data_downsampled,
normalized_test_data_downsampled, y_train, y_test, 'linear')
evaluation(down_svm[0], down_svm[1], y_train, y_test)

# Running the SVM with 'rbf' kernel with downsampled data.
down_svm = svm_experiment(normalized_train_data_downsampled,
normalized_test_data_downsampled, y_train, y_test, 'rbf')
evaluation(down_svm[0], down_svm[1], y_train, y_test)
```

```
Evaluation Report:
Training Accuracy: 1.000
Testing Accuracy: 0.593
Training F1 Score: 0.999
Testing F1 Score: 0.591
Training Precision: 1.000
Testing Precision: 0.599
Training Recall: 1.000
Testing Recall: 0.593
Overfitting Check: 0.407
Significant overfitting detected. Model may need more regularization.
```



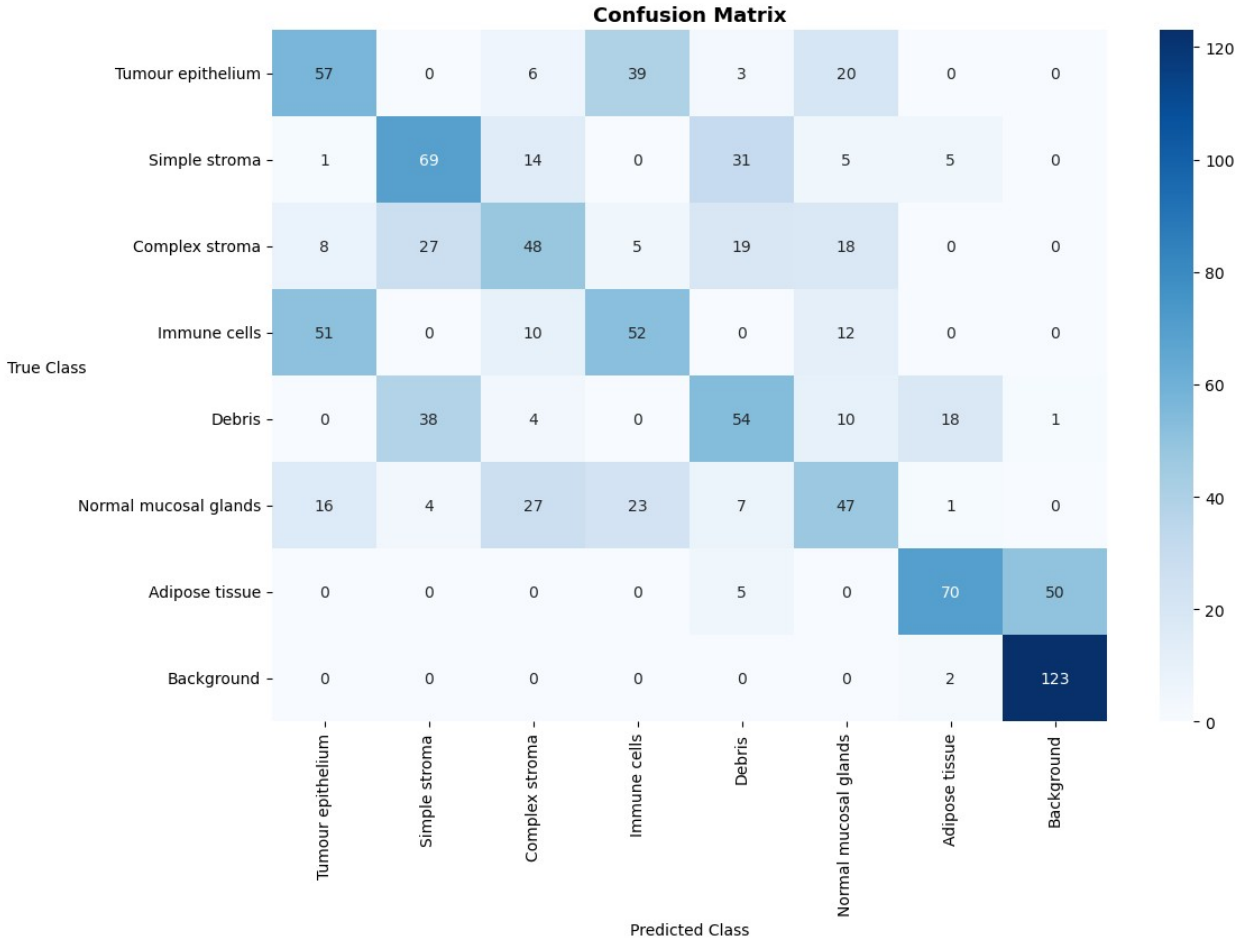
```
Evaluation Report:
Training Accuracy: 0.867
Testing Accuracy: 0.727
Training F1 Score: 0.867
Testing F1 Score: 0.727
Training Precision: 0.872
Testing Precision: 0.740
Training Recall: 0.867
Testing Recall: 0.727
Overfitting Check: 0.140
Slight overfitting detected.
```




```
# Running a softmax experiment with downsampled data.
down_softmax = softmax_experiment(normalized_train_data_downsampled,
normalized_test_data_downsampled, y_train, y_test)
evaluation(down_softmax[0], down_softmax[1], y_train, y_test)
```

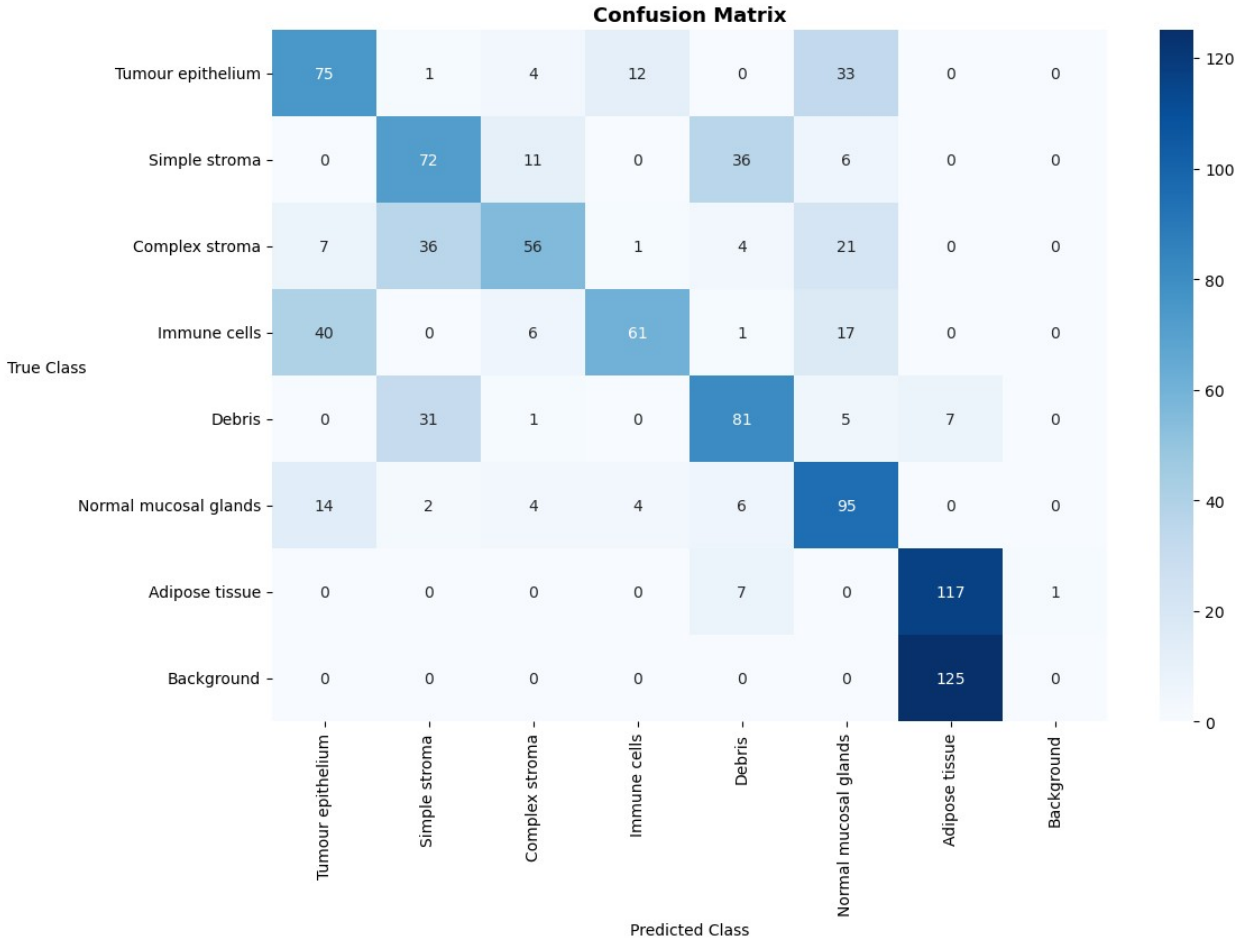
```
D:\Anaconda\Lib\site-packages\sklearn\linear_model\_logistic.py:469:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

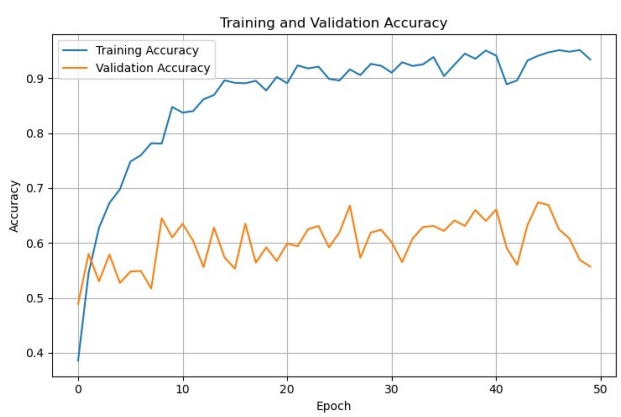
```
Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(

Evaluation Report:
Training Accuracy: 0.999
Testing Accuracy: 0.520
Training F1 Score: 0.998
Testing F1 Score: 0.512
Training Precision: 0.999
Testing Precision: 0.514
Training Recall: 0.999
Testing Recall: 0.520
Overfitting Check: 0.479
Significant overfitting detected. Model may need more regularization.
```

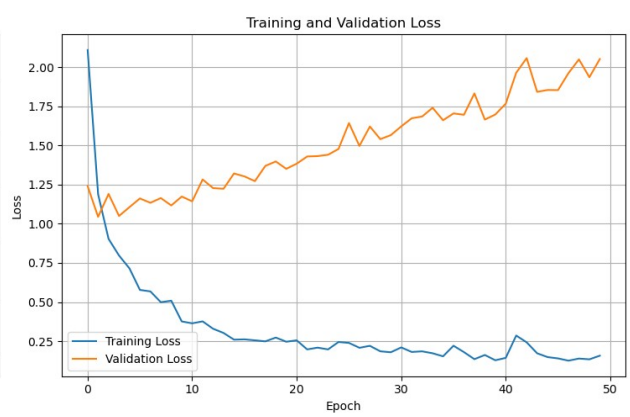


```python
# Running a NN experiment with downsampled data.
down_nn = nn_experiment(normalized_train_data_downsampled,
normalized_test_data_downsampled, y_train, y_test)
evaluation(down_nn[0], down_nn[1], y_train, y_test)
plot_vgg_history(down_nn[2])
```

```
D:\Anaconda\Lib\site-packages\keras\src\layers\core\dense.py:95:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Evaluation Report:
Training Accuracy: 0.872
Testing Accuracy: 0.557
Training F1 Score: 0.831
Testing F1 Score: 0.517
Training Precision: 0.934
Testing Precision: 0.516
Training Recall: 0.872
Testing Recall: 0.557
Overfitting Check: 0.315
Significant overfitting detected. Model may need more regularization.
```



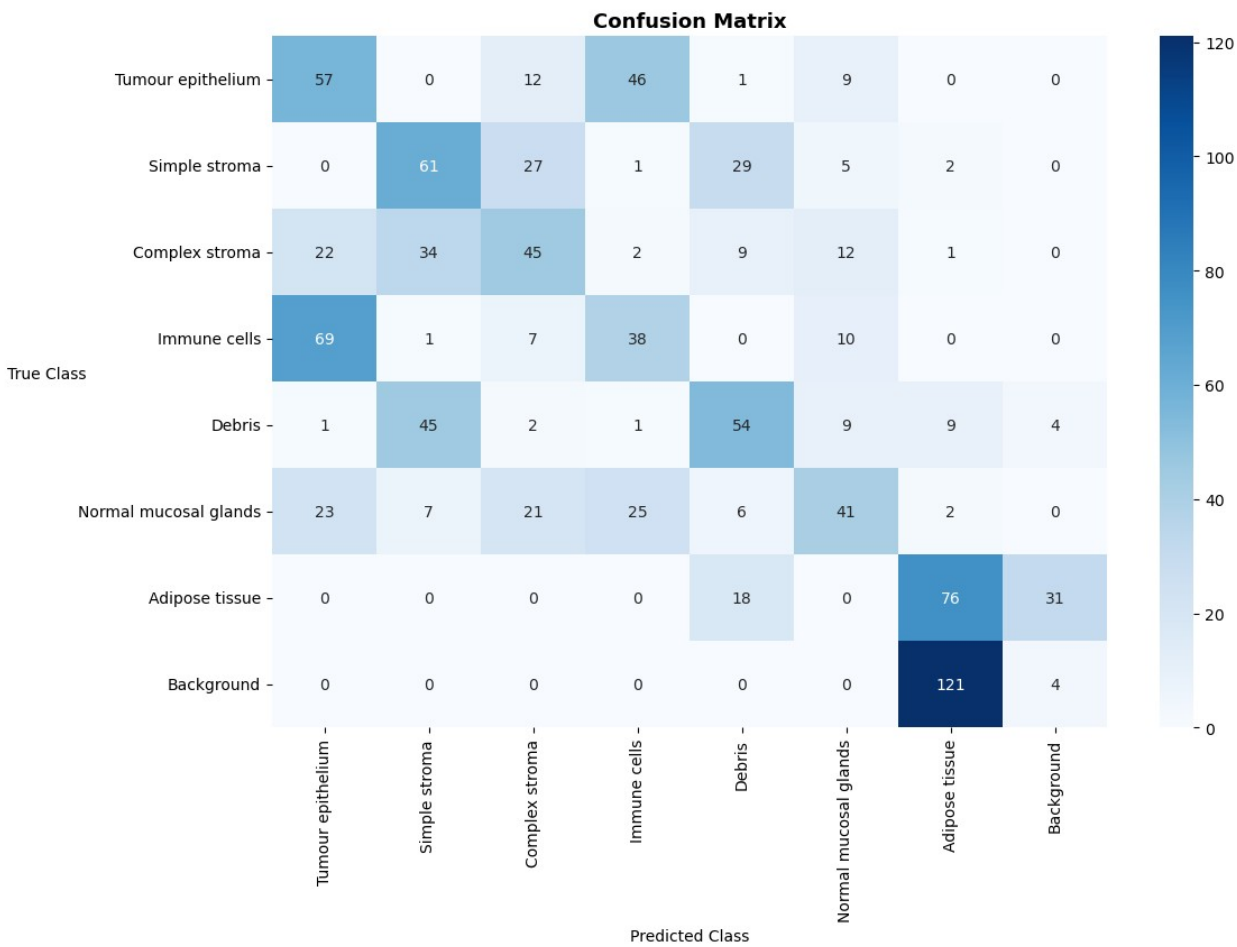





```
# Running the SVM with 'linear' kernel with PCA extracted data.
pca_svm = svm_experiment(normalized_train_data_pca,
normalized_test_data_pca, y_train, y_test, 'linear')
evaluation(pca_svm[0], pca_svm[1], y_train, y_test)

# Running the SVM with 'rbf' kernel with PCA extracted data.
pca_svm = svm_experiment(normalized_train_data_pca,
```

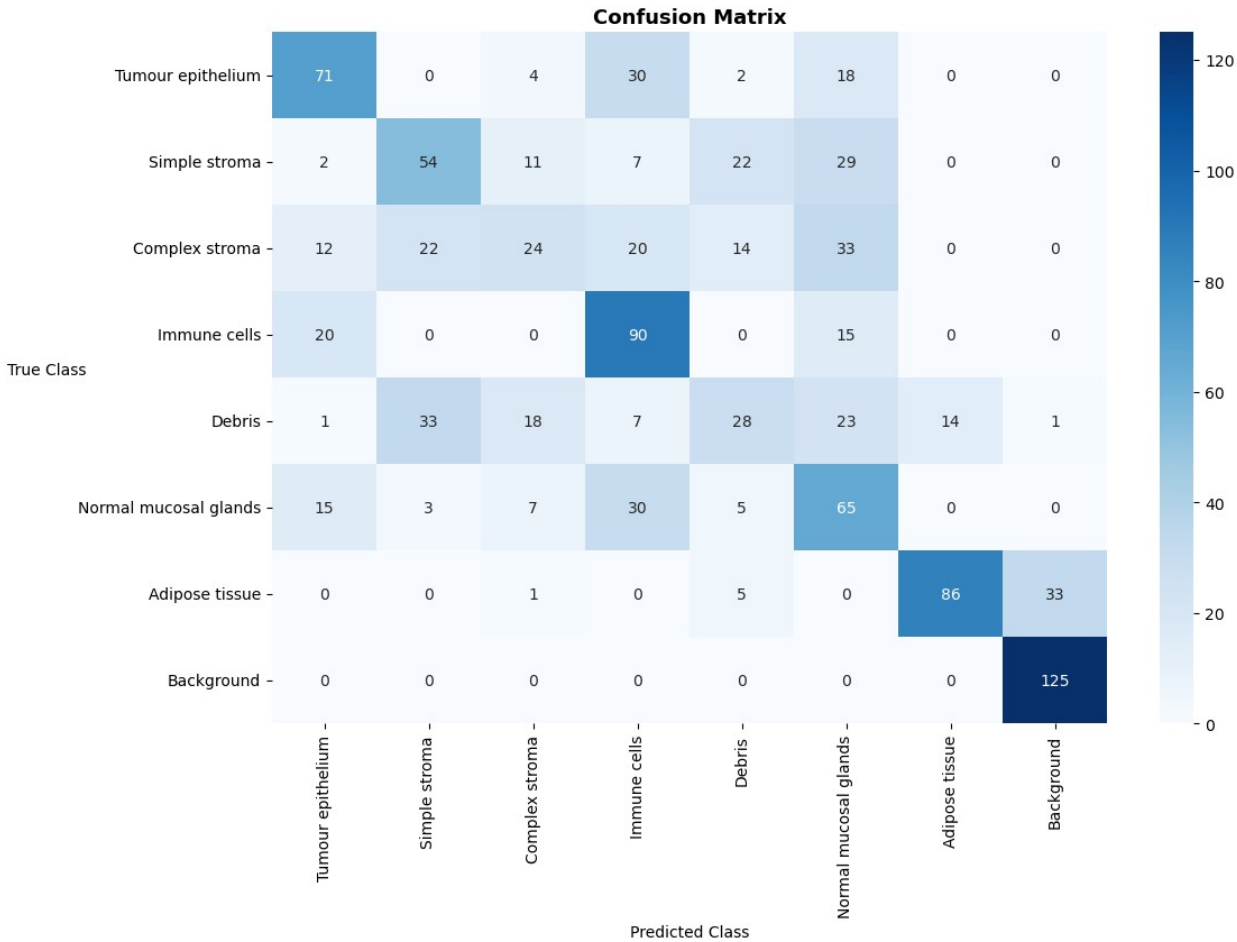
```
normalized_test_data_pca, y_train, y_test, 'rbf')
evaluation(pca_svm[0], pca_svm[1], y_train, y_test)
```

```
Evaluation Report:
Training Accuracy: 0.884
Testing Accuracy: 0.376
Training F1 Score: 0.884
Testing F1 Score: 0.358
Training Precision: 0.885
Testing Precision: 0.359
Training Recall: 0.884
Testing Recall: 0.376
Overfitting Check: 0.508
Significant overfitting detected. Model may need more regularization.
```




```
Evaluation Report:
Training Accuracy: 0.952
Testing Accuracy: 0.543
Training F1 Score: 0.952
Testing F1 Score: 0.527
```

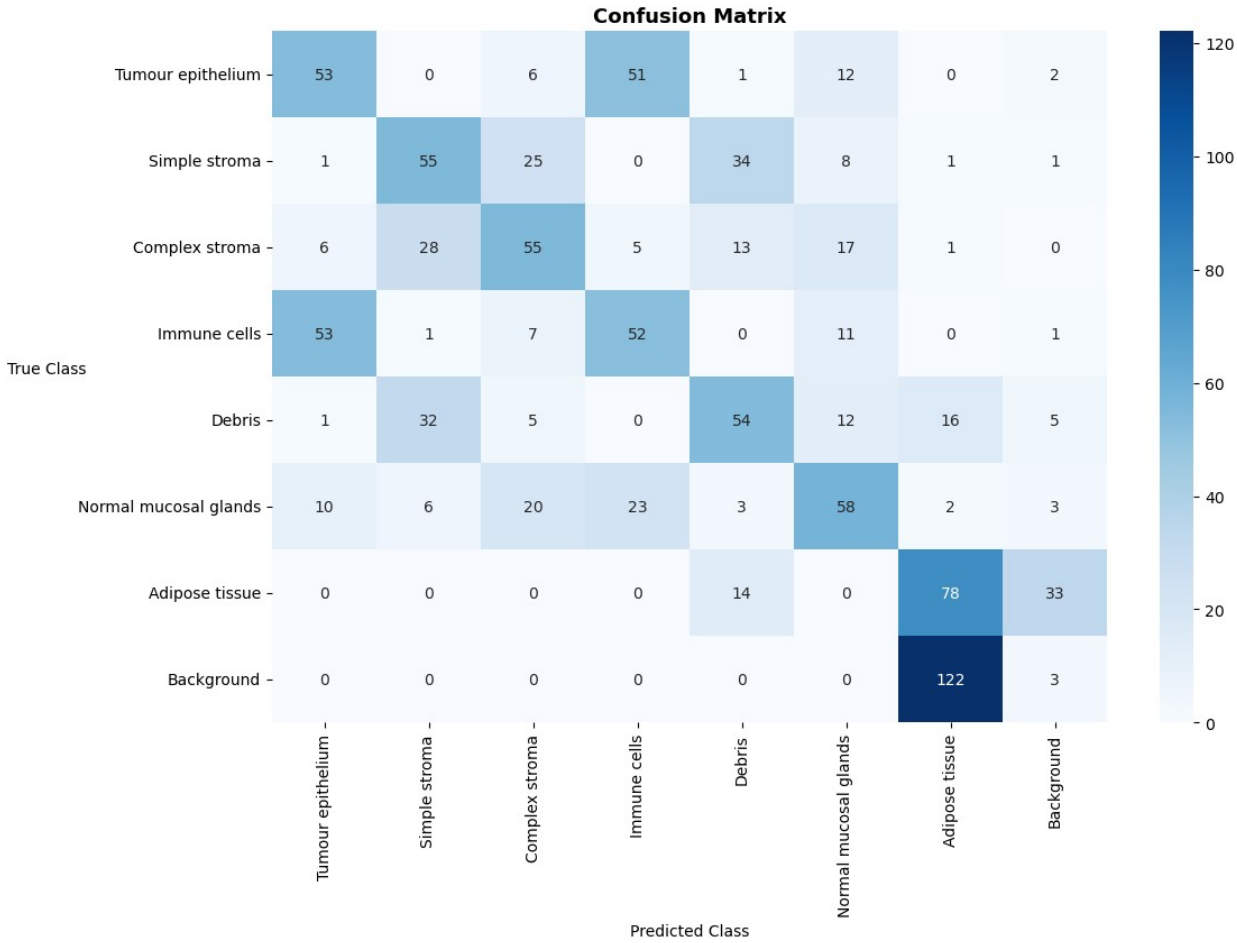
```
Training Precision: 0.954
Testing Precision: 0.537
Training Recall: 0.952
Testing Recall: 0.543
Overfitting Check: 0.409
Significant overfitting detected. Model may need more regularization.
```




```
# Running the softmax experiment with PCA extracted data.
pca_softmax = softmax_experiment(normalized_train_data_pca,
normalized_test_data_pca, y_train, y_test)
evaluation(pca_softmax[0], pca_softmax[1], y_train, y_test)
```

```
Evaluation Report:
Training Accuracy: 0.787
Testing Accuracy: 0.408
Training F1 Score: 0.786
Testing F1 Score: 0.392
Training Precision: 0.787
Testing Precision: 0.388
Training Recall: 0.787
```

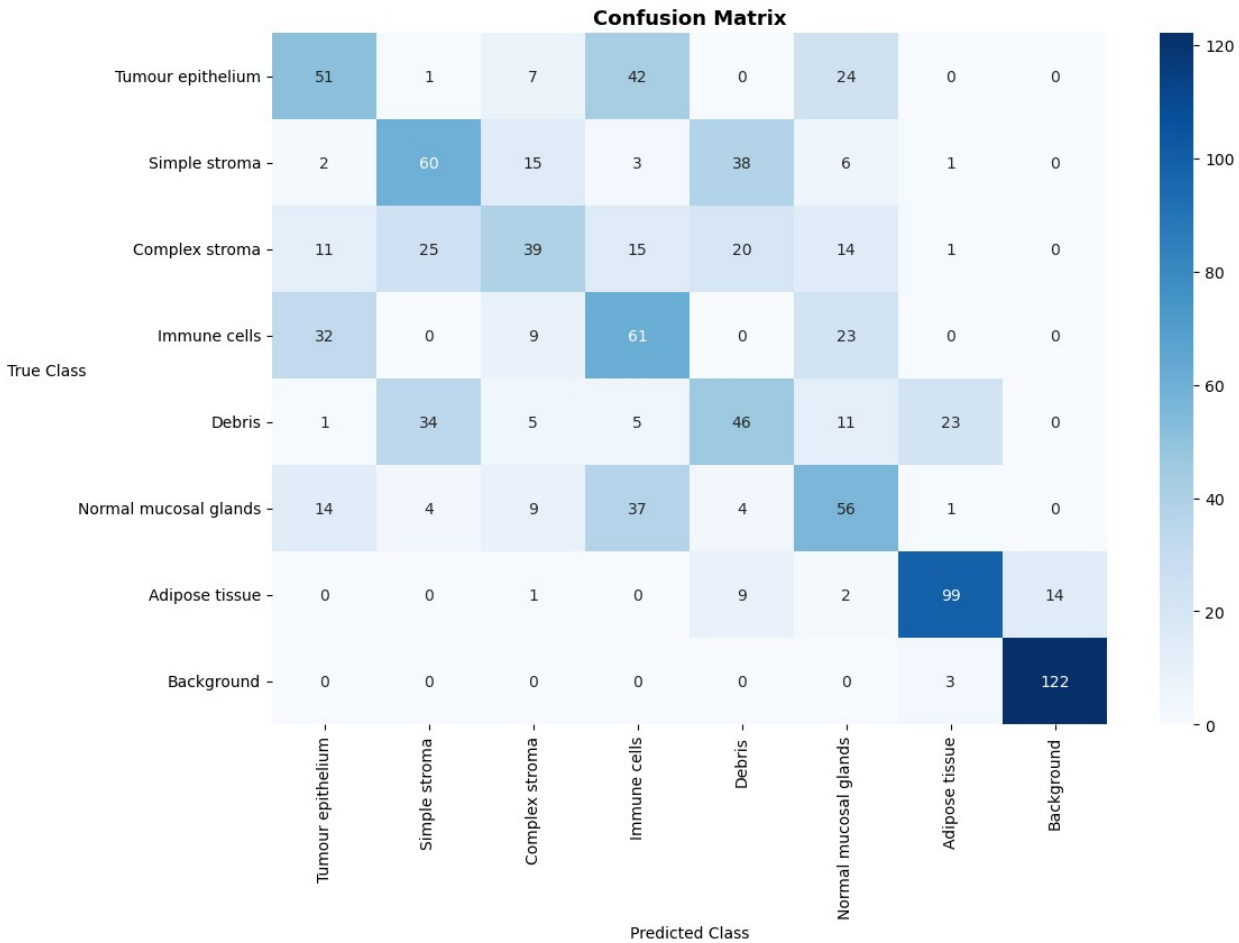
```
Testing Recall: 0.408
Overfitting Check: 0.379
Significant overfitting detected. Model may need more regularization.
```

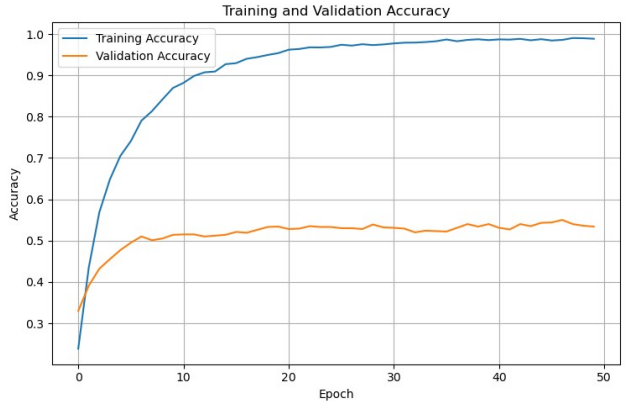


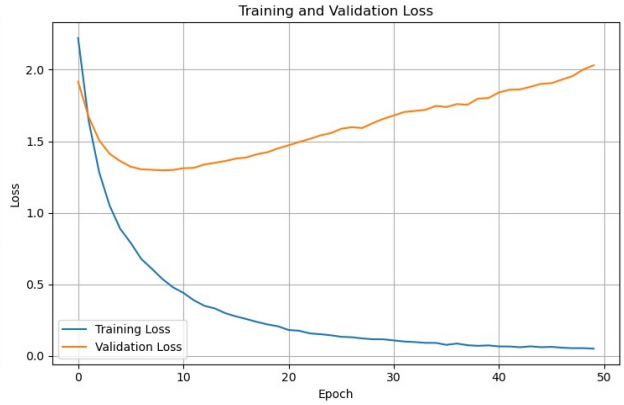

```
# Running the NN Experiment with with PCA extracted data.
pca_nn = nn_experiment(normalized_train_data_pca,
normalized_test_data_pca, y_train, y_test)
evaluation(pca_nn[0], pca_nn[1], y_train, y_test)
plot_vgg_history(pca_nn[2])
```

```
D:\Anaconda\Lib\site-packages\keras\src\layers\core\dense.py:95:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Evaluation Report:
Training Accuracy: 0.998
Testing Accuracy: 0.534
Training F1 Score: 0.998
```

```
Testing F1 Score: 0.529
Training Precision: 0.998
Testing Precision: 0.531
Training Recall: 0.998
Testing Recall: 0.534
Overfitting Check: 0.464
Significant overfitting detected. Model may need more regularization.
```

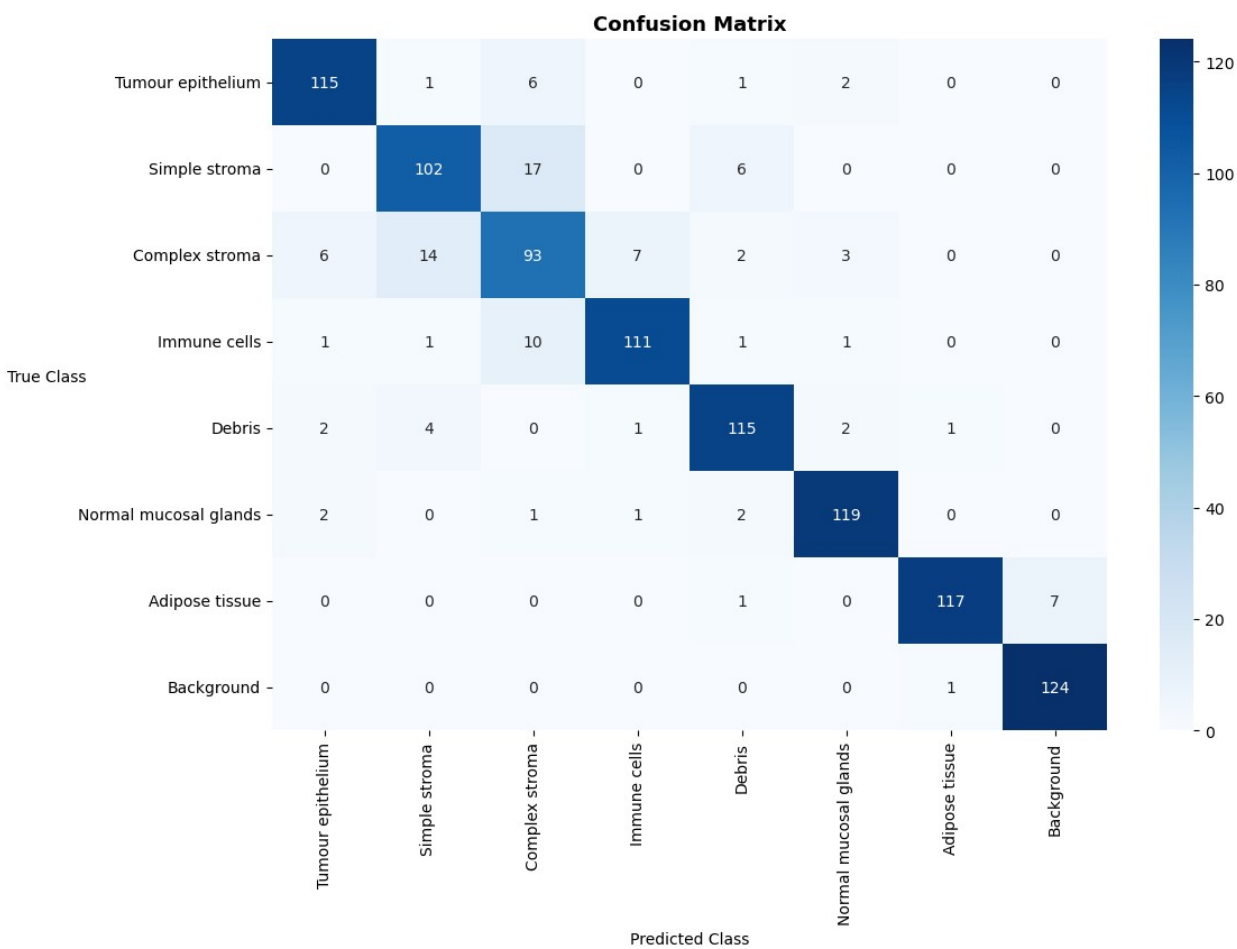




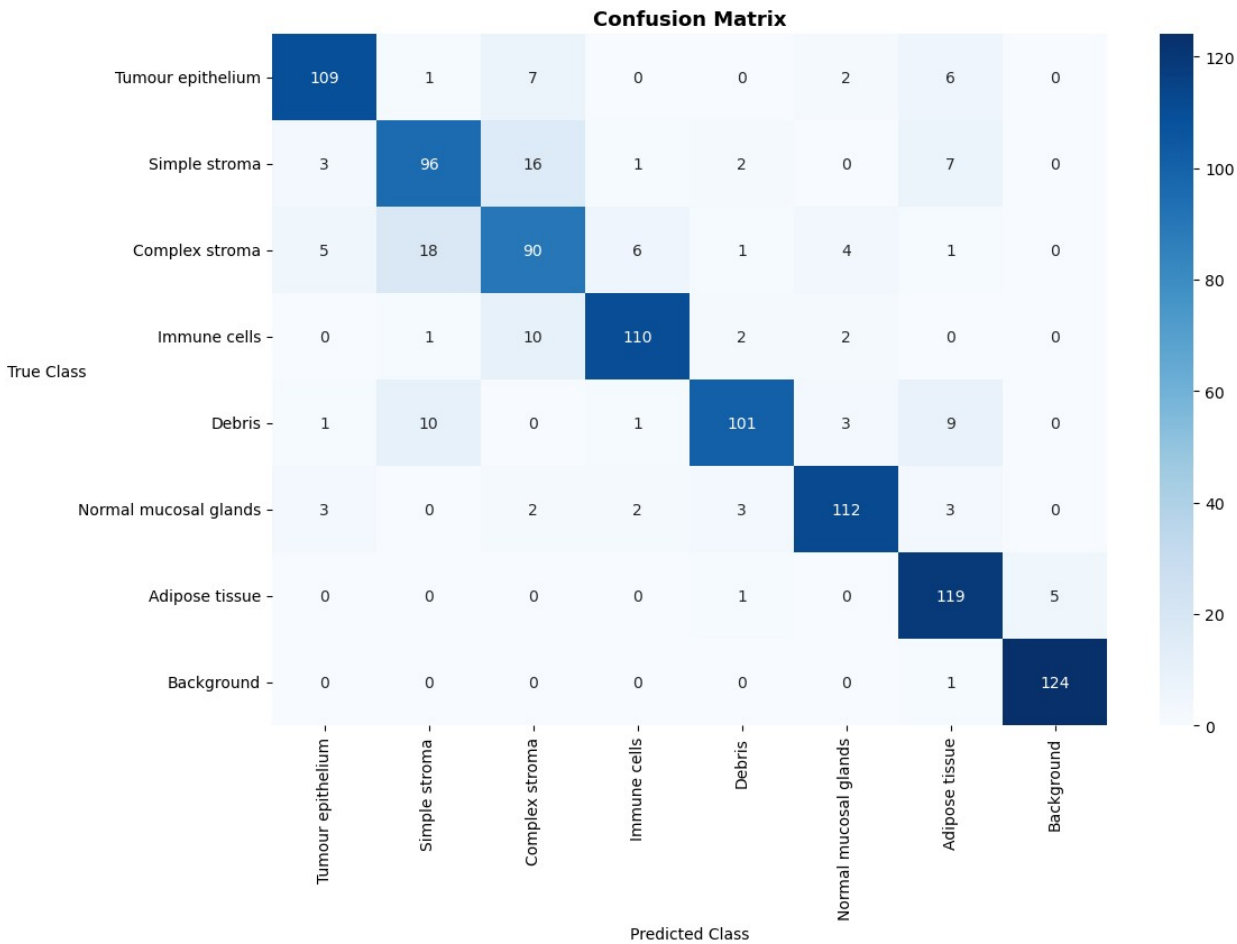


```python
# Running the SVM with 'linear' kernel with VGG extracted data.
vgg_svm = svm_experiment(normalized_train_data_vgg,
normalized_test_data_vgg, y_train, y_test, 'linear')
evaluation(vgg_svm[0], vgg_svm[1], y_train, y_test)

# Running the SVM with 'rbf' kernel with VGG extracted data.
vgg_svm = svm_experiment(normalized_train_data_vgg,
normalized_test_data_vgg, y_train, y_test, 'rbf')
evaluation(vgg_svm[0], vgg_svm[1], y_train, y_test)
```

```
Evaluation Report:
Training Accuracy: 1.000
Testing Accuracy: 0.896
Training F1 Score: 1.000
Testing F1 Score: 0.896
Training Precision: 1.000
Testing Precision: 0.896
Training Recall: 1.000
Testing Recall: 0.896
Overfitting Check: 0.104
Slight overfitting detected.
```



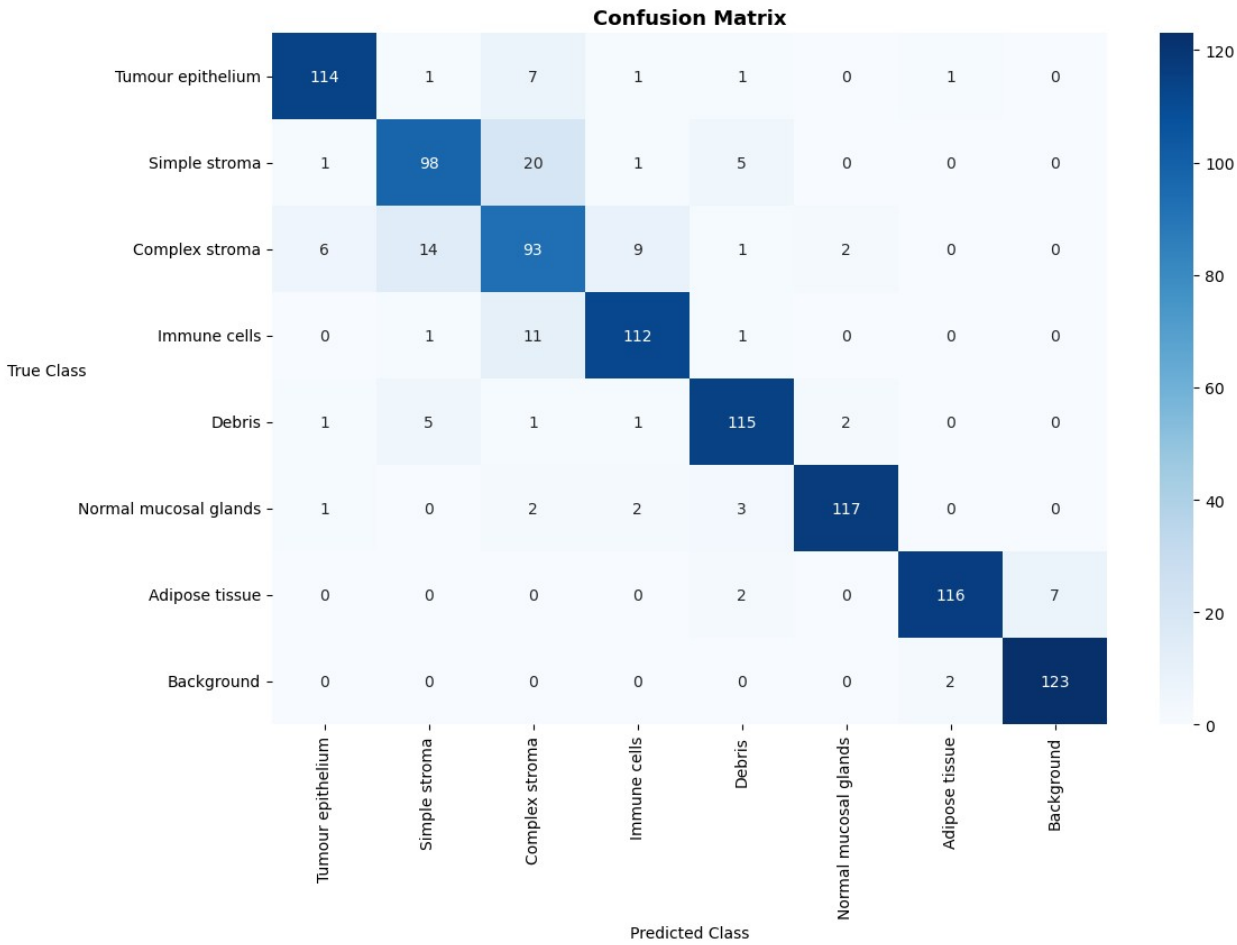
```
Evaluation Report:
Training Accuracy: 0.981
Testing Accuracy: 0.861
Training F1 Score: 0.981
Testing F1 Score: 0.861
Training Precision: 0.981
Testing Precision: 0.863
Training Recall: 0.981
Testing Recall: 0.861
Overfitting Check: 0.120
Slight overfitting detected.
```




```
# Running the Softmax experiment with VGG extracted data.
vgg_softmax = softmax_experiment(normalized_train_data_vgg,
normalized_test_data_vgg, y_train, y_test)
evaluation(vgg_softmax[0], vgg_softmax[1], y_train, y_test)
```

```
Evaluation Report:
Training Accuracy: 1.000
Testing Accuracy: 0.888
```

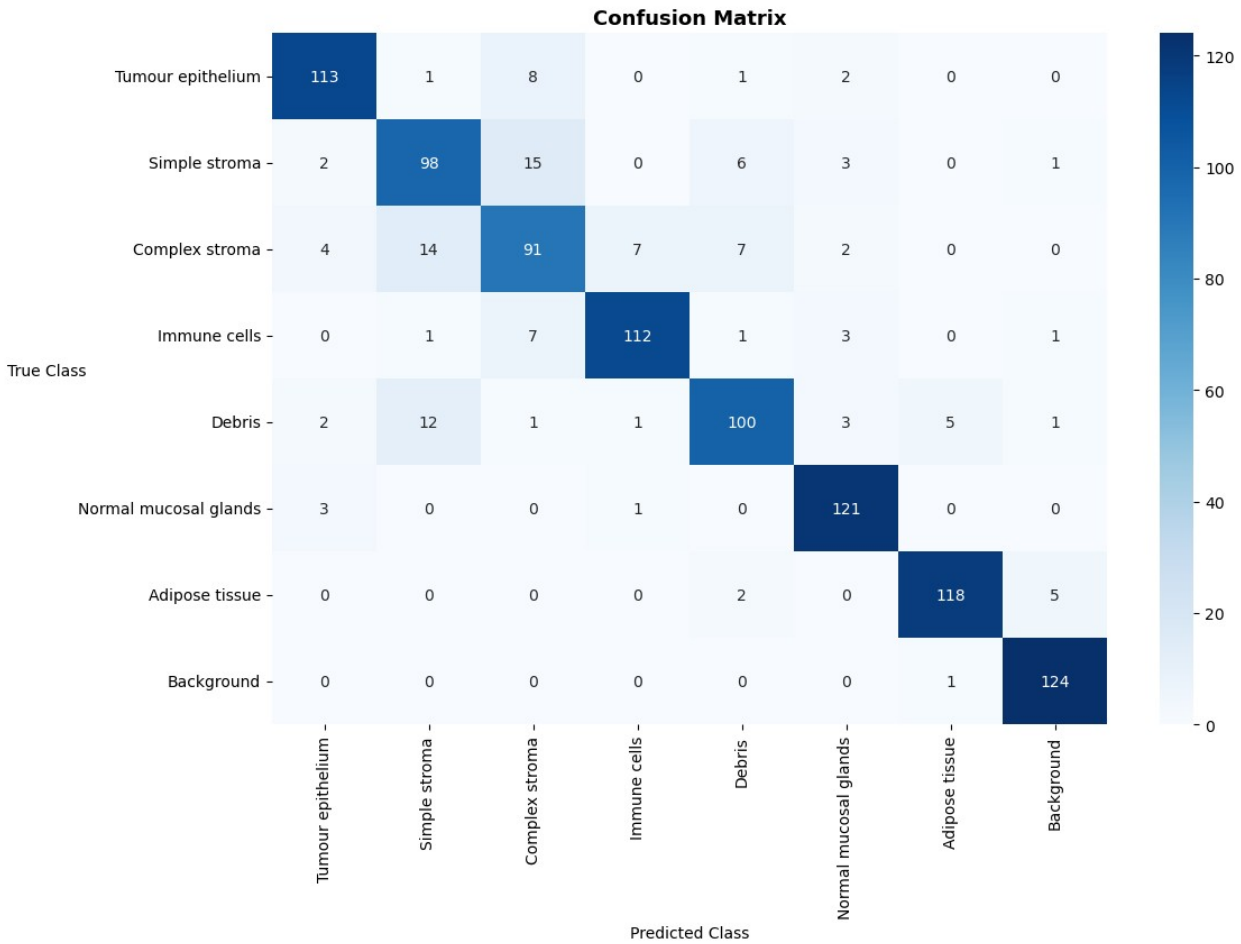
```
Training F1 Score: 1.000
Testing F1 Score: 0.889
Training Precision: 1.000
Testing Precision: 0.890
Training Recall: 1.000
Testing Recall: 0.888
Overfitting Check: 0.112
Slight overfitting detected.
```

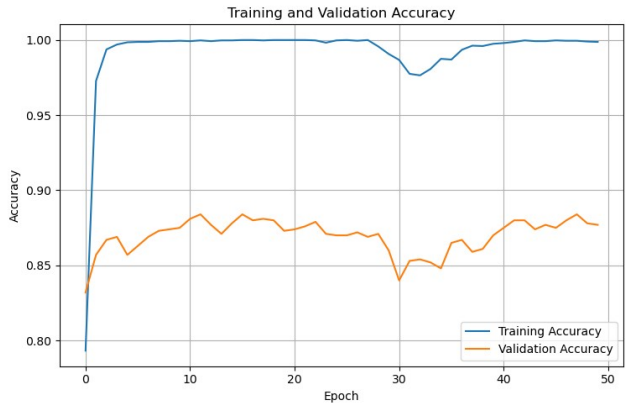


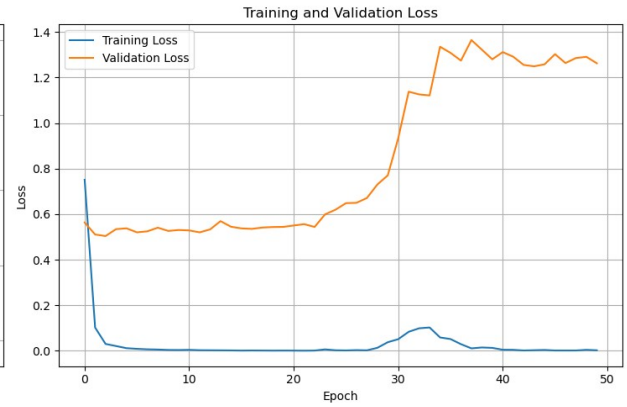

```
# Running the NN Experiment with VGG extracted data.
vgg_nn = nn_experiment(normalized_train_data_vgg,
normalized_test_data_vgg, y_train, y_test)
evaluation(vgg_nn[0], vgg_nn[1], y_train, y_test)
plot_vgg_history(vgg_nn[2])
```

```
D:\Anaconda\Lib\site-packages\keras\src\layers\core\dense.py:95:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Evaluation Report:
Training Accuracy: 1.000
Testing Accuracy: 0.877
Training F1 Score: 1.000
Testing F1 Score: 0.876
Training Precision: 1.000
Testing Precision: 0.876
Training Recall: 1.000
Testing Recall: 0.877
Overfitting Check: 0.123
Slight overfitting detected.
```







Experiemnts Conclusions:

Comparison of Feature Extraction Methods:

Generally, *VGG-16* features consistently yielded the best performance across all classifiers, indicating the effectiveness of Transfer Learning in image classification tasks.

Comparison of Classifiers:

Neural networks showed better performance than *SVM* and *Softmax* in most cases, especially when combined with VGG-16 features.

Overfitting:

Monitoring the difference between training and testing accuracy is important for identifying overfitting. In some cases, it might be necessary to tune hyperparameters or use additional regularization techniques.

Advanced Experiment:

As per previous conclusions, we will try to improve the VGG16 + NN experiment to achieve higher accuracy and F1-score, thus making the model more reliable.

We will use a *Data Augmentation* technique, which creates new variations of training images. This will include rotation, shifts, zoom, and brightness changes using the `ImageDataGenerator` function.

Additionally, we will implement a Fine-Tuning method. Since *VGG16* is adaptable and flexible, we will *fine-tune* the model by adjusting the weights of the last layers to better suit the dataset. This technique enables the model to learn higher-level features relevant to the specific histology classification task. The `learning_rate` will be set to `0.00001` to avoid overwriting the pre-trained layers and their weights.

Besides these methods, we will also set up several callbacks for the model:

- **EarlyStopping** - which prevents over-training and over-fitting by stopping the training prior compliting all the epochs.
- **ReduceLROnPlateau** - Atomatic configuration which reduces the learning rate when a metric pleatu is spotted - allows to model to acheive a global minimum.
- **ModelCheckpoint** - Ensures that we save the best performing model on the validation set.

```
# Setting up the data augmentation technique.
data_gen = ImageDataGenerator( # Initialize ImageDataGenerator for
data augmentation.
            rotation_range=45, # Random rotations up to 45 degrees
            width_shift_range=0.3, # Horizontal shifts up to 30% of
the width
            height_shift_range=0.3, # Vertical shifts up to 30% of the
height
            zoom_range=0.4, # Random zoom up to 40%
            shear_range=0.3, # Shearing transformations
            brightness_range=[0.6, 1.4], # Adjust brightness
            horizontal_flip=True, # Random horizontal flips
            vertical_flip=True, # Random vertical flips
            fill_mode='reflect', # Fill empty areas with reflected
pixels
            )

# Loading the VGG16 base model with ImageNet weights.
base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(150, 150, 3)) # Removing the top classification layer and
setting input shape as 150x150x3.
base_model.trainable = True  #Set the base model to be trainable
initially.
for layer in base_model.layers[:-4]: # Iterate through layers,
freezing all but the last 4 layers.
  layer.trainable = False  # Set layer to non-trainable.
print(f"  VGG-16: {len(base_model.layers) - sum([1 for layer in
base_model.layers if layer.trainable])} layers frozen, {sum([1 for
layer in base_model.layers if layer.trainable])} layers unfrozen for
fine-tuning.")
```

```
  VGG-16: 15 layers frozen, 4 layers unfrozen for fine-tuning.
```

```
# Create a new model by adding custom classification layers on top of
VGG-16
model = models.Sequential([ # Initialize a Sequential model.
    base_model, # Add the VGG-16 base model.
    layers.GlobalAveragePooling2D(), # Add a GlobalAveragePooling2D
layer to reduce spatial dimensions.
    layers.Dense(256, activation='relu'), # Add a dense layer with 256
neurons and ReLU activation.
```

```
    layers.Dropout(0.5), # Add a dropout layer with a rate of 0.5 for
regularization.
    layers.Dense(len(class_names), activation='softmax'), # Add the
output layer with Softmax activation.
        ])

# Compile the model with a very low learning rate
# A low learning rate (0.00001) is critical for fine-tuning
# to avoid destroying the pre-trained weights.
model.compile( # Compile the model.
    optimizer=Adam(learning_rate=0.00001), # Use Adam optimizer with a
very low learning rate for fine-tuning.
    loss='categorical_crossentropy', # Set the loss function.
    metrics=['accuracy'] # Set the evaluation metric.
) # Print a summary of the model architecture, including trainable
parameters.
 model.summary()

# Callbacks for improved training

# EarlyStopping: Stops training if validation loss does not improve.
early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True, verbose=1) # Initialize EarlyStopping
callback.
# ReduceLROnPlateau: Reduces the learning rate if the monitored metric
does not improve.
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
patience=5, min_lr=0.000001, verbose=1) # Initialize ReduceLROnPlateau
callback.
# ModelCheckpoint: Saves the best model based on validation accuracy.
checkpoint_filepath = 'best_model_exp10.keras' # Define the file path
for saving the best model.
model_checkpoint_callback = ModelCheckpoint( # Initialize
ModelCheckpoint callback.
filepath=checkpoint_filepath, # Path to save the model.
save_weights_only=False, # Save the entire model (architecture +
weights).
monitor='val_accuracy', # Metric to monitor for saving the best model.
mode='max', # Save when the monitored metric is maximized.
save_best_only=True, # Only save the best model found so far.
verbose=1 # Show verbose output during saving.
)

# Group all callbacks into a list.
callbacks = [early_stopping, reduce_lr, model_checkpoint_callback]

# Train with augmented data
y_train_cat = to_categorical(y_train, len(class_names)) # Convert
training labels to one-hot encoding.
```

```
y_test_cat = to_categorical(y_test, len(class_names)) # Convert
testing labels to one-hot encoding.

# Begining to train the model.
history = model.fit( # Train the model using the data generator for
augmented data.
    data_gen.flow(x_train, y_train_cat, batch_size=64), # Use
augmented training data
    validation_data=(x_test, y_test_cat), # Use standard test data for
validation
    epochs=50, # Max epochs (EarlyStopping will likely stop it sooner)
    steps_per_epoch=len(x_train) // 32, # Number of batches per epoch
    callbacks=callbacks, # Apply all control mechanisms
    verbose=1
)
```

```
Epoch 1/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.1734 - loss:
4.8170

D:\Anaconda\Lib\site-packages\keras\src\trainers\
epoch_iterator.py:116: UserWarning: Your input ran out of data;
interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches. You may need to
use the `.repeat()` function when building your dataset.
  self._interrupted_warning()


Epoch 1: val_accuracy improved from None to 0.39500, saving model to
best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 131s 1s/step - accuracy: 0.1982 - loss:
3.3898 - val_accuracy: 0.3950 - val_loss: 1.6535 - learning_rate:
1.0000e-05
Epoch 2/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.3040 - loss:
1.9117
Epoch 2: val_accuracy improved from 0.39500 to 0.56800, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.3230 - loss:
1.8504 - val_accuracy: 0.5680 - val_loss: 1.3106 - learning_rate:
1.0000e-05
Epoch 3/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.4183 - loss:
1.6015
Epoch 3: val_accuracy improved from 0.56800 to 0.63000, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 132s 1s/step - accuracy: 0.4473 - loss:
1.5471 - val_accuracy: 0.6300 - val_loss: 1.1034 - learning_rate:
1.0000e-05
Epoch 4/50
```

```
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.5204 - loss:
1.3635
Epoch 4: val_accuracy improved from 0.63000 to 0.71000, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.5288 - loss:
1.3541 - val_accuracy: 0.7100 - val_loss: 0.9049 - learning_rate:
1.0000e-05
Epoch 5/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.5547 - loss:
1.2607
Epoch 5: val_accuracy improved from 0.71000 to 0.74900, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.5748 - loss:
1.2202 - val_accuracy: 0.7490 - val_loss: 0.7728 - learning_rate:
1.0000e-05
Epoch 6/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.6109 - loss:
1.1452
Epoch 6: val_accuracy improved from 0.74900 to 0.78100, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.6220 - loss:
1.1113 - val_accuracy: 0.7810 - val_loss: 0.6582 - learning_rate:
1.0000e-05
Epoch 7/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.6532 - loss:
1.0198
Epoch 7: val_accuracy improved from 0.78100 to 0.81200, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.6675 - loss:
0.9936 - val_accuracy: 0.8120 - val_loss: 0.5531 - learning_rate:
1.0000e-05
Epoch 8/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.6964 - loss:
0.9056
Epoch 8: val_accuracy improved from 0.81200 to 0.83100, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7075 - loss:
0.8756 - val_accuracy: 0.8310 - val_loss: 0.4980 - learning_rate:
1.0000e-05
Epoch 9/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7428 - loss:
0.7819
Epoch 9: val_accuracy improved from 0.83100 to 0.85600, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7398 - loss:
0.7836 - val_accuracy: 0.8560 - val_loss: 0.4291 - learning_rate:
1.0000e-05
Epoch 10/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7425 - loss:
```

```
0.7567
Epoch 10: val_accuracy improved from 0.85600 to 0.86300, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7560 - loss:
0.7361 - val_accuracy: 0.8630 - val_loss: 0.4087 - learning_rate:
1.0000e-05
Epoch 11/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7570 - loss:
0.7492
Epoch 11: val_accuracy improved from 0.86300 to 0.86500, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7590 - loss:
0.7236 - val_accuracy: 0.8650 - val_loss: 0.4019 - learning_rate:
1.0000e-05
Epoch 12/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7754 - loss:
0.6609
Epoch 12: val_accuracy improved from 0.86500 to 0.87800, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7793 - loss:
0.6663 - val_accuracy: 0.8780 - val_loss: 0.3707 - learning_rate:
1.0000e-05
Epoch 13/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7816 - loss:
0.6315
Epoch 13: val_accuracy improved from 0.87800 to 0.89700, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.7815 - loss:
0.6548 - val_accuracy: 0.8970 - val_loss: 0.3219 - learning_rate:
1.0000e-05
Epoch 14/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8002 - loss:
0.6124
Epoch 14: val_accuracy did not improve from 0.89700
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8075 - loss:
0.5987 - val_accuracy: 0.8970 - val_loss: 0.3150 - learning_rate:
1.0000e-05
Epoch 15/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8095 - loss:
0.5738
Epoch 15: val_accuracy improved from 0.89700 to 0.90800, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8058 - loss:
0.5970 - val_accuracy: 0.9080 - val_loss: 0.2960 - learning_rate:
1.0000e-05
Epoch 16/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.7993 - loss:
0.6013
Epoch 16: val_accuracy did not improve from 0.90800
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8085 - loss:
0.5767 - val_accuracy: 0.9050 - val_loss: 0.2821 - learning_rate:
1.0000e-05
Epoch 17/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8243 - loss:
0.5350
Epoch 17: val_accuracy did not improve from 0.90800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8255 - loss:
0.5377 - val_accuracy: 0.9070 - val_loss: 0.2783 - learning_rate:
1.0000e-05
Epoch 18/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8290 - loss:
0.5130
Epoch 18: val_accuracy improved from 0.90800 to 0.91600, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8235 - loss:
0.5259 - val_accuracy: 0.9160 - val_loss: 0.2768 - learning_rate:
1.0000e-05
Epoch 19/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8314 - loss:
0.5110
Epoch 19: val_accuracy did not improve from 0.91600
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8307 - loss:
0.5081 - val_accuracy: 0.9130 - val_loss: 0.2609 - learning_rate:
1.0000e-05
Epoch 20/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8277 - loss:
0.5115
Epoch 20: val_accuracy did not improve from 0.91600
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8307 - loss:
0.5202 - val_accuracy: 0.9160 - val_loss: 0.2594 - learning_rate:
1.0000e-05
Epoch 21/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8322 - loss:
0.4836
Epoch 21: val_accuracy did not improve from 0.91600
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8397 - loss:
0.4789 - val_accuracy: 0.9130 - val_loss: 0.2701 - learning_rate:
1.0000e-05
Epoch 22/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8399 - loss:
0.4974
Epoch 22: val_accuracy did not improve from 0.91600
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8320 - loss:
0.4965 - val_accuracy: 0.9150 - val_loss: 0.2664 - learning_rate:
1.0000e-05
Epoch 23/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8425 - loss:
0.4740
```

```
Epoch 23: val_accuracy did not improve from 0.91600
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8497 - loss:
0.4609 - val_accuracy: 0.9100 - val_loss: 0.2810 - learning_rate:
1.0000e-05
Epoch 24/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8450 - loss:
0.4840
Epoch 24: val_accuracy improved from 0.91600 to 0.92800, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8525 - loss:
0.4622 - val_accuracy: 0.9280 - val_loss: 0.2368 - learning_rate:
1.0000e-05
Epoch 25/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8497 - loss:
0.4405
Epoch 25: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8482 - loss:
0.4481 - val_accuracy: 0.9240 - val_loss: 0.2329 - learning_rate:
1.0000e-05
Epoch 26/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8516 - loss:
0.4552
Epoch 26: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8562 - loss:
0.4473 - val_accuracy: 0.9230 - val_loss: 0.2351 - learning_rate:
1.0000e-05
Epoch 27/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8563 - loss:
0.4327
Epoch 27: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8553 - loss:
0.4437 - val_accuracy: 0.9230 - val_loss: 0.2364 - learning_rate:
1.0000e-05
Epoch 28/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8690 - loss:
0.4194
Epoch 28: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8645 - loss:
0.4395 - val_accuracy: 0.9210 - val_loss: 0.2440 - learning_rate:
1.0000e-05
Epoch 29/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8517 - loss:
0.4374
Epoch 29: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8525 - loss:
0.4280 - val_accuracy: 0.9200 - val_loss: 0.2376 - learning_rate:
1.0000e-05
Epoch 30/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8646 - loss:
```

```
0.4060
Epoch 30: ReduceLROnPlateau reducing learning rate to
1.9999999494757505e-06.

Epoch 30: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8630 - loss:
0.4161 - val_accuracy: 0.9190 - val_loss: 0.2466 - learning_rate:
1.0000e-05
Epoch 31/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8572 - loss:
0.4198
Epoch 31: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8595 - loss:
0.4040 - val_accuracy: 0.9240 - val_loss: 0.2286 - learning_rate:
2.0000e-06
Epoch 32/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8721 - loss:
0.3967
Epoch 32: val_accuracy did not improve from 0.92800
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8683 - loss:
0.4108 - val_accuracy: 0.9280 - val_loss: 0.2315 - learning_rate:
2.0000e-06
Epoch 33/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8702 - loss:
0.3934
Epoch 33: val_accuracy improved from 0.92800 to 0.93000, saving model
to best_model_exp10.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8720 - loss:
0.3902 - val_accuracy: 0.9300 - val_loss: 0.2240 - learning_rate:
2.0000e-06
Epoch 34/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8673 - loss:
0.4066
Epoch 34: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8733 - loss:
0.3949 - val_accuracy: 0.9270 - val_loss: 0.2232 - learning_rate:
2.0000e-06
Epoch 35/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8692 - loss:
0.4050
Epoch 35: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8695 - loss:
0.4058 - val_accuracy: 0.9230 - val_loss: 0.2229 - learning_rate:
2.0000e-06
Epoch 36/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8708 - loss:
0.4078
Epoch 36: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8712 - loss:
```

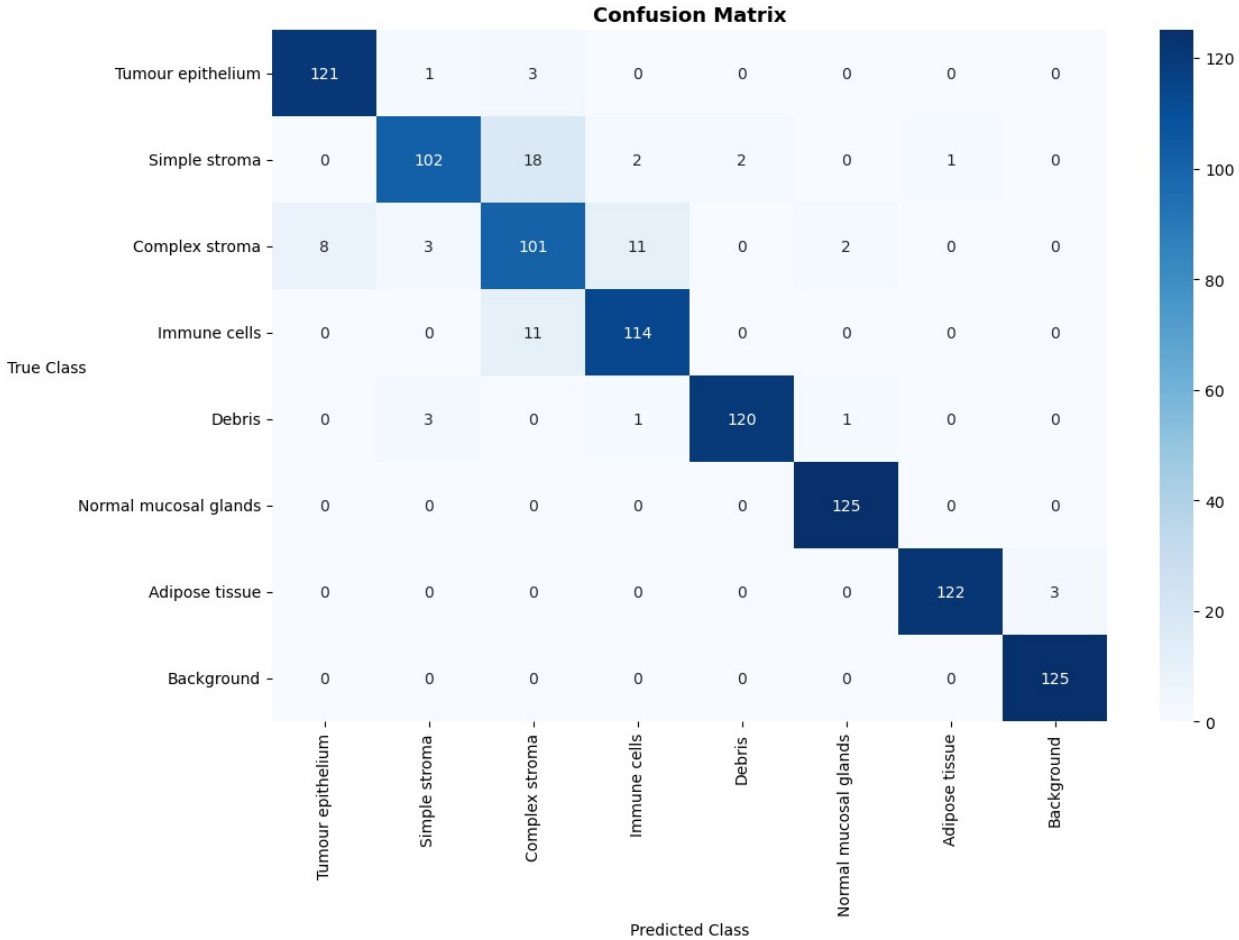
```
0.3954 - val_accuracy: 0.9270 - val_loss: 0.2252 - learning_rate:
2.0000e-06
Epoch 37/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8728 - loss:
0.4054
Epoch 37: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8748 - loss:
0.3926 - val_accuracy: 0.9290 - val_loss: 0.2257 - learning_rate:
2.0000e-06
Epoch 38/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8667 - loss:
0.4256
Epoch 38: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8692 - loss:
0.4133 - val_accuracy: 0.9270 - val_loss: 0.2167 - learning_rate:
2.0000e-06
Epoch 39/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8688 - loss:
0.3930
Epoch 39: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8675 - loss:
0.3987 - val_accuracy: 0.9240 - val_loss: 0.2230 - learning_rate:
2.0000e-06
Epoch 40/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8773 - loss:
0.3944
Epoch 40: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 131s 1s/step - accuracy: 0.8752 - loss:
0.4025 - val_accuracy: 0.9230 - val_loss: 0.2217 - learning_rate:
2.0000e-06
Epoch 41/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8840 - loss:
0.3702
Epoch 41: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 130s 1s/step - accuracy: 0.8808 - loss:
0.3758 - val_accuracy: 0.9260 - val_loss: 0.2171 - learning_rate:
2.0000e-06
Epoch 42/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8753 - loss:
0.3758
Epoch 42: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8745 - loss:
0.3895 - val_accuracy: 0.9220 - val_loss: 0.2272 - learning_rate:
2.0000e-06
Epoch 43/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8786 - loss:
0.3746
Epoch 43: ReduceLROnPlateau reducing learning rate to 1e-06.
```

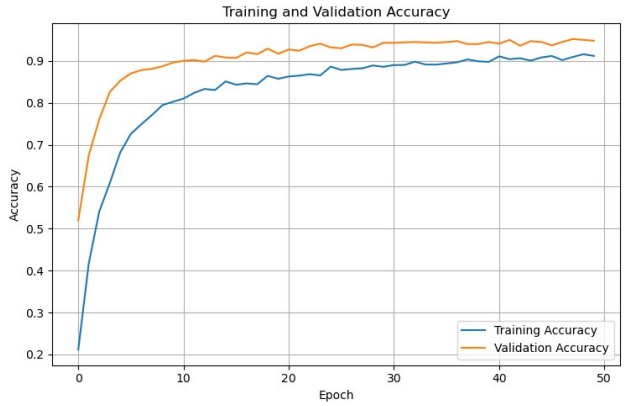
```
Epoch 43: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8745 - loss:
0.3779 - val_accuracy: 0.9260 - val_loss: 0.2175 - learning_rate:
2.0000e-06
Epoch 44/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8729 - loss:
0.4043
Epoch 44: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8737 - loss:
0.4014 - val_accuracy: 0.9280 - val_loss: 0.2150 - learning_rate:
1.0000e-06
Epoch 45/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8726 - loss:
0.3946
Epoch 45: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8773 - loss:
0.3830 - val_accuracy: 0.9240 - val_loss: 0.2199 - learning_rate:
1.0000e-06
Epoch 46/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8907 - loss:
0.3385
Epoch 46: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8813 - loss:
0.3636 - val_accuracy: 0.9270 - val_loss: 0.2179 - learning_rate:
1.0000e-06
Epoch 47/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8748 - loss:
0.4059
Epoch 47: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8727 - loss:
0.4043 - val_accuracy: 0.9280 - val_loss: 0.2142 - learning_rate:
1.0000e-06
Epoch 48/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8746 - loss:
0.3776
Epoch 48: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8745 - loss:
0.3798 - val_accuracy: 0.9230 - val_loss: 0.2119 - learning_rate:
1.0000e-06
Epoch 49/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:44 2s/step - accuracy: 0.8827 - loss:
0.3630
Epoch 49: val_accuracy did not improve from 0.93000
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8758 - loss:
0.3726 - val_accuracy: 0.9270 - val_loss: 0.2128 - learning_rate:
1.0000e-06
Epoch 50/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 1:43 2s/step - accuracy: 0.8759 - loss:
0.3753
Epoch 50: val_accuracy did not improve from 0.93000
```

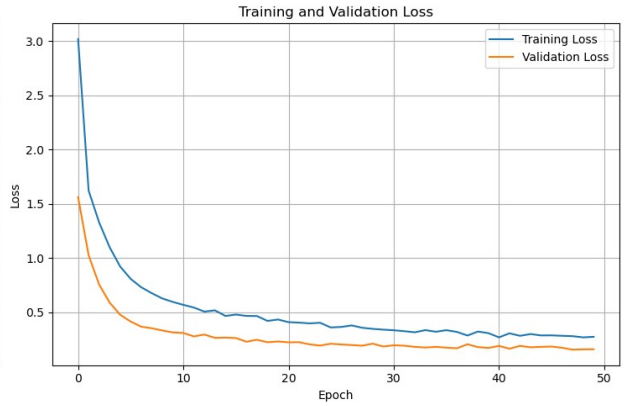
```
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.8785 - loss:
0.3708 - val_accuracy: 0.9280 - val_loss: 0.2141 - learning_rate:
1.0000e-06
Restoring model weights from the end of the best epoch: 48.
```

```python
# Load the  model for evaluation.
best_model = tf.keras.models.load_model(checkpoint_filepath)

#Evaluation procces.
print("10th Experiment Model evaluation")

y_pred_train = np.argmax(best_model.predict(x_train), axis=1)
y_pred_test = np.argmax(best_model.predict(x_test), axis=1)

# Launching evalutaton functions as used for pervious experiments.
results = evaluation(y_pred_train, y_pred_test, y_train, y_test)
plot_vgg_history(history)
```

```
10th Experiment Model evaluation
125/125 ━━━━━━━━━━━━━━━━━━━━ 91s 726ms/step
32/32 ━━━━━━━━━━━━━━━━━━━━ 23s 714ms/step
Evaluation Report:
Training Accuracy: 0.927
Testing Accuracy: 0.930
Training F1 Score: 0.928
Testing F1 Score: 0.930
Training Precision: 0.928
Testing Precision: 0.932
Training Recall: 0.927
Testing Recall: 0.930
Overfitting Check: -0.003
Excellent generalization - Test performance matches or exceeds
training.
```







Advanced Experiment: 1st Attempt Improvement

Since the first try was based on `150x150` pixel input, We would try now to input `224x224` pixels. We suspect that increase the details, would assist the attempt to improve the accuracy on *Simple* and *Complex Stroma* as they are our wickness.

```
#Setting the training and test images from 150x150 pixels to 224x224
pixels.
x_train_resized = tf.image.resize(x_train, [224, 224]).numpy()
x_test_resized = tf.image.resize(x_test, [224, 224]).numpy()
x_train_resized = x_train_resized.astype('float32')
x_test_resized = x_test_resized.astype('float32')

# Create a new model by adding custom classification layers on top of
VGG-16 as same as previous model but different input shapes.
base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
for layer in base_model.layers[:-4]:
  layer.trainable = False
print(f"  VGG-16: {len(base_model.layers) - sum([1 for layer in
base_model.layers if layer.trainable])} layers frozen, {sum([1 for
layer in base_model.layers if layer.trainable])} layers unfrozen for
fine-tuning.")
```

```
  VGG-16: 15 layers frozen, 4 layers unfrozen for fine-tuning.
```

```
# Create another model by adding custom classification layers on top
of VGG-16 same as before.
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(len(class_names), activation='softmax'),
])

# Compile the model with a very low learning rate
# A low learning rate (0.00001) is critical for fine-tuning
# to avoid destroying the pre-trained weights.
model.compile(
    optimizer=Adam(learning_rate=0.00001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Same callbacks as before to improve training.
# EarlyStopping, ReudceLROnPlateau and checkpoint_filepath.
early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True, verbose=1) # Initialize EarlyStopping
callback.
# ReduceLROnPlateau: Reduces the learning rate if the monitored metric
does not improve.
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
patience=5, min_lr=0.000001, verbose=1) # Initialize ReduceLROnPlateau
callback.
# ModelCheckpoint: Saves the best model based on validation accuracy.
```

```
checkpoint_filepath = 'best_model_exp10_224.keras' # Define the file
path for saving the best model.
model_checkpoint_callback = ModelCheckpoint( # Initialize
ModelCheckpoint callback.
filepath=checkpoint_filepath, # Path to save the model.
save_weights_only=False, # Save the entire model (architecture +
weights).
monitor='val_accuracy', # Metric to monitor for saving the best model.
mode='max', # Save when the monitored metric is maximized.
save_best_only=True, # Only save the best model found so far.
verbose=1 # Show verbose output during saving.
)

# Group all callbacks into a list same as before.
callbacks = [early_stopping, reduce_lr, model_checkpoint_callback]

# Begining to train the model.
history = model.fit(
    data_gen.flow(x_train_resized, y_train_cat, batch_size=64), # Use
augmented training data
    validation_data=(x_test_resized, y_test_cat), # Use standard test
data for validation
    epochs=50, # Max epochs (EarlyStopping will likely stop it sooner)
    steps_per_epoch=len(x_train_resized) // 32, # Number of batches
per epoch
    callbacks=callbacks, # Apply all control mechanisms
    verbose=1
)
```

```
Epoch 1/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:46 4s/step - accuracy: 0.1618 - loss:
4.5285

D:\Anaconda\Lib\site-packages\keras\src\trainers\
epoch_iterator.py:116: UserWarning: Your input ran out of data;
interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches. You may need to
use the `.repeat()` function when building your dataset.
  self._interrupted_warning()


Epoch 1: val_accuracy improved from None to 0.52000, saving model to
best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 284s 2s/step - accuracy: 0.2115 - loss:
3.0194 - val_accuracy: 0.5200 - val_loss: 1.5598 - learning_rate:
1.0000e-05
Epoch 2/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:55 4s/step - accuracy: 0.3755 - loss:
1.7199
Epoch 2: val_accuracy improved from 0.52000 to 0.67500, saving model
```

```
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 291s 2s/step - accuracy: 0.4178 - loss:
1.6206 - val_accuracy: 0.6750 - val_loss: 1.0244 - learning_rate:
1.0000e-05
Epoch 3/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:58 4s/step - accuracy: 0.5170 - loss:
1.3874
Epoch 3: val_accuracy improved from 0.67500 to 0.76100, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 296s 2s/step - accuracy: 0.5415 - loss:
1.3271 - val_accuracy: 0.7610 - val_loss: 0.7539 - learning_rate:
1.0000e-05
Epoch 4/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:53 4s/step - accuracy: 0.5858 - loss:
1.1544
Epoch 4: val_accuracy improved from 0.76100 to 0.82600, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 291s 2s/step - accuracy: 0.6090 - loss:
1.0978 - val_accuracy: 0.8260 - val_loss: 0.5871 - learning_rate:
1.0000e-05
Epoch 5/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:54 4s/step - accuracy: 0.6558 - loss:
0.9569
Epoch 5: val_accuracy improved from 0.82600 to 0.85300, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 290s 2s/step - accuracy: 0.6825 - loss:
0.9198 - val_accuracy: 0.8530 - val_loss: 0.4752 - learning_rate:
1.0000e-05
Epoch 6/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.7136 - loss:
0.8261
Epoch 6: val_accuracy improved from 0.85300 to 0.87000, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.7260 - loss:
0.8061 - val_accuracy: 0.8700 - val_loss: 0.4123 - learning_rate:
1.0000e-05
Epoch 7/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:53 4s/step - accuracy: 0.7474 - loss:
0.7318
Epoch 7: val_accuracy improved from 0.87000 to 0.87800, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 290s 2s/step - accuracy: 0.7490 - loss:
0.7289 - val_accuracy: 0.8780 - val_loss: 0.3653 - learning_rate:
1.0000e-05
Epoch 8/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.7660 - loss:
0.6844
Epoch 8: val_accuracy improved from 0.87800 to 0.88100, saving model
to best_model_exp10_224.keras
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 288s 2s/step - accuracy: 0.7707 - loss:
0.6742 - val_accuracy: 0.8810 - val_loss: 0.3509 - learning_rate:
1.0000e-05
Epoch 9/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:49 4s/step - accuracy: 0.7992 - loss:
0.6079
Epoch 9: val_accuracy improved from 0.88100 to 0.88700, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.7945 - loss:
0.6258 - val_accuracy: 0.8870 - val_loss: 0.3303 - learning_rate:
1.0000e-05
Epoch 10/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8009 - loss:
0.5963
Epoch 10: val_accuracy improved from 0.88700 to 0.89600, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8025 - loss:
0.5937 - val_accuracy: 0.8960 - val_loss: 0.3124 - learning_rate:
1.0000e-05
Epoch 11/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:54 4s/step - accuracy: 0.8136 - loss:
0.5706
Epoch 11: val_accuracy improved from 0.89600 to 0.90000, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 291s 2s/step - accuracy: 0.8100 - loss:
0.5668 - val_accuracy: 0.9000 - val_loss: 0.3072 - learning_rate:
1.0000e-05
Epoch 12/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:55 4s/step - accuracy: 0.8306 - loss:
0.5230
Epoch 12: val_accuracy improved from 0.90000 to 0.90200, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 292s 2s/step - accuracy: 0.8235 - loss:
0.5421 - val_accuracy: 0.9020 - val_loss: 0.2756 - learning_rate:
1.0000e-05
Epoch 13/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8374 - loss:
0.5034
Epoch 13: val_accuracy did not improve from 0.90200
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8330 - loss:
0.5044 - val_accuracy: 0.8980 - val_loss: 0.2928 - learning_rate:
1.0000e-05
Epoch 14/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8321 - loss:
0.5209
Epoch 14: val_accuracy improved from 0.90200 to 0.91200, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 288s 2s/step - accuracy: 0.8305 - loss:
0.5160 - val_accuracy: 0.9120 - val_loss: 0.2622 - learning_rate:
```

```
1.0000e-05
Epoch 15/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8530 - loss:
0.4671
Epoch 15: val_accuracy did not improve from 0.91200
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8510 - loss:
0.4644 - val_accuracy: 0.9080 - val_loss: 0.2647 - learning_rate:
1.0000e-05
Epoch 16/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8353 - loss:
0.4919
Epoch 16: val_accuracy did not improve from 0.91200
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8430 - loss:
0.4771 - val_accuracy: 0.9070 - val_loss: 0.2604 - learning_rate:
1.0000e-05
Epoch 17/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8514 - loss:
0.4699
Epoch 17: val_accuracy improved from 0.91200 to 0.92000, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8462 - loss:
0.4648 - val_accuracy: 0.9200 - val_loss: 0.2265 - learning_rate:
1.0000e-05
Epoch 18/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8385 - loss:
0.4737
Epoch 18: val_accuracy did not improve from 0.92000
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8443 - loss:
0.4635 - val_accuracy: 0.9160 - val_loss: 0.2446 - learning_rate:
1.0000e-05
Epoch 19/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8625 - loss:
0.4337
Epoch 19: val_accuracy improved from 0.92000 to 0.92900, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8643 - loss:
0.4185 - val_accuracy: 0.9290 - val_loss: 0.2217 - learning_rate:
1.0000e-05
Epoch 20/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8620 - loss:
0.4431
Epoch 20: val_accuracy did not improve from 0.92900
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8572 - loss:
0.4315 - val_accuracy: 0.9170 - val_loss: 0.2288 - learning_rate:
1.0000e-05
Epoch 21/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8576 - loss:
0.4085
Epoch 21: val_accuracy did not improve from 0.92900
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8627 - loss:
0.4070 - val_accuracy: 0.9270 - val_loss: 0.2212 - learning_rate:
1.0000e-05
Epoch 22/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8593 - loss:
0.4129
Epoch 22: val_accuracy did not improve from 0.92900
125/125 ━━━━━━━━━━━━━━━━━━━━ 285s 2s/step - accuracy: 0.8648 - loss:
0.4029 - val_accuracy: 0.9240 - val_loss: 0.2222 - learning_rate:
1.0000e-05
Epoch 23/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:52 4s/step - accuracy: 0.8652 - loss:
0.4094
Epoch 23: val_accuracy improved from 0.92900 to 0.93500, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 290s 2s/step - accuracy: 0.8683 - loss:
0.3957 - val_accuracy: 0.9350 - val_loss: 0.2020 - learning_rate:
1.0000e-05
Epoch 24/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:58 4s/step - accuracy: 0.8625 - loss:
0.3988
Epoch 24: val_accuracy improved from 0.93500 to 0.94100, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 296s 2s/step - accuracy: 0.8652 - loss:
0.4014 - val_accuracy: 0.9410 - val_loss: 0.1909 - learning_rate:
1.0000e-05
Epoch 25/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8794 - loss:
0.3664
Epoch 25: val_accuracy did not improve from 0.94100
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8863 - loss:
0.3585 - val_accuracy: 0.9320 - val_loss: 0.2078 - learning_rate:
1.0000e-05
Epoch 26/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8701 - loss:
0.3668
Epoch 26: val_accuracy did not improve from 0.94100
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8783 - loss:
0.3628 - val_accuracy: 0.9300 - val_loss: 0.2011 - learning_rate:
1.0000e-05
Epoch 27/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:50 4s/step - accuracy: 0.8730 - loss:
0.3903
Epoch 27: val_accuracy did not improve from 0.94100
125/125 ━━━━━━━━━━━━━━━━━━━━ 286s 2s/step - accuracy: 0.8808 - loss:
0.3768 - val_accuracy: 0.9390 - val_loss: 0.1960 - learning_rate:
1.0000e-05
Epoch 28/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:51 4s/step - accuracy: 0.8773 - loss:
```

```
0.3707
Epoch 28: val_accuracy did not improve from 0.94100
125/125 ━━━━━━━━━━━━━━━━━━━━ 287s 2s/step - accuracy: 0.8823 - loss:
0.3554 - val_accuracy: 0.9380 - val_loss: 0.1898 - learning_rate:
1.0000e-05
Epoch 29/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:52 4s/step - accuracy: 0.8867 - loss:
0.3580
Epoch 29: val_accuracy did not improve from 0.94100
125/125 ━━━━━━━━━━━━━━━━━━━━ 290s 2s/step - accuracy: 0.8890 - loss:
0.3452 - val_accuracy: 0.9320 - val_loss: 0.2087 - learning_rate:
1.0000e-05
Epoch 30/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:59 4s/step - accuracy: 0.8823 - loss:
0.3480
Epoch 30: val_accuracy improved from 0.94100 to 0.94300, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 298s 2s/step - accuracy: 0.8857 - loss:
0.3381 - val_accuracy: 0.9430 - val_loss: 0.1824 - learning_rate:
1.0000e-05
Epoch 31/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 4:00 4s/step - accuracy: 0.8966 - loss:
0.3278
Epoch 31: val_accuracy did not improve from 0.94300
125/125 ━━━━━━━━━━━━━━━━━━━━ 299s 2s/step - accuracy: 0.8900 - loss:
0.3323 - val_accuracy: 0.9430 - val_loss: 0.1937 - learning_rate:
1.0000e-05
Epoch 32/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:57 4s/step - accuracy: 0.8896 - loss:
0.3229
Epoch 32: val_accuracy improved from 0.94300 to 0.94400, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 295s 2s/step - accuracy: 0.8903 - loss:
0.3237 - val_accuracy: 0.9440 - val_loss: 0.1899 - learning_rate:
1.0000e-05
Epoch 33/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:58 4s/step - accuracy: 0.9020 - loss:
0.3035
Epoch 33: val_accuracy improved from 0.94400 to 0.94500, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 291s 2s/step - accuracy: 0.8980 - loss:
0.3136 - val_accuracy: 0.9450 - val_loss: 0.1791 - learning_rate:
1.0000e-05
Epoch 34/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:33 3s/step - accuracy: 0.8971 - loss:
0.3204
Epoch 34: val_accuracy did not improve from 0.94500
125/125 ━━━━━━━━━━━━━━━━━━━━ 264s 2s/step - accuracy: 0.8913 - loss:
0.3334 - val_accuracy: 0.9440 - val_loss: 0.1734 - learning_rate:
```

```
1.0000e-05
Epoch 35/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:32 3s/step - accuracy: 0.8936 - loss:
0.3107
Epoch 35: val_accuracy did not improve from 0.94500
125/125 ━━━━━━━━━━━━━━━━━━━━ 264s 2s/step - accuracy: 0.8910 - loss:
0.3187 - val_accuracy: 0.9430 - val_loss: 0.1788 - learning_rate:
1.0000e-05
Epoch 36/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:32 3s/step - accuracy: 0.8912 - loss:
0.3369
Epoch 36: val_accuracy did not improve from 0.94500
125/125 ━━━━━━━━━━━━━━━━━━━━ 262s 2s/step - accuracy: 0.8938 - loss:
0.3328 - val_accuracy: 0.9450 - val_loss: 0.1722 - learning_rate:
1.0000e-05
Epoch 37/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:31 3s/step - accuracy: 0.8892 - loss:
0.3309
Epoch 37: val_accuracy improved from 0.94500 to 0.94700, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 262s 2s/step - accuracy: 0.8965 - loss:
0.3172 - val_accuracy: 0.9470 - val_loss: 0.1659 - learning_rate:
1.0000e-05
Epoch 38/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:33 3s/step - accuracy: 0.9047 - loss:
0.2760
Epoch 38: val_accuracy did not improve from 0.94700
125/125 ━━━━━━━━━━━━━━━━━━━━ 264s 2s/step - accuracy: 0.9035 - loss:
0.2836 - val_accuracy: 0.9400 - val_loss: 0.2030 - learning_rate:
1.0000e-05
Epoch 39/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:36 3s/step - accuracy: 0.9020 - loss:
0.3208
Epoch 39: val_accuracy did not improve from 0.94700
125/125 ━━━━━━━━━━━━━━━━━━━━ 267s 2s/step - accuracy: 0.8990 - loss:
0.3205 - val_accuracy: 0.9400 - val_loss: 0.1768 - learning_rate:
1.0000e-05
Epoch 40/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:35 3s/step - accuracy: 0.9013 - loss:
0.3023
Epoch 40: val_accuracy did not improve from 0.94700
125/125 ━━━━━━━━━━━━━━━━━━━━ 268s 2s/step - accuracy: 0.8975 - loss:
0.3057 - val_accuracy: 0.9450 - val_loss: 0.1699 - learning_rate:
1.0000e-05
Epoch 41/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:34 3s/step - accuracy: 0.9078 - loss:
0.2695
Epoch 41: val_accuracy did not improve from 0.94700
125/125 ━━━━━━━━━━━━━━━━━━━━ 267s 2s/step - accuracy: 0.9107 - loss:
```

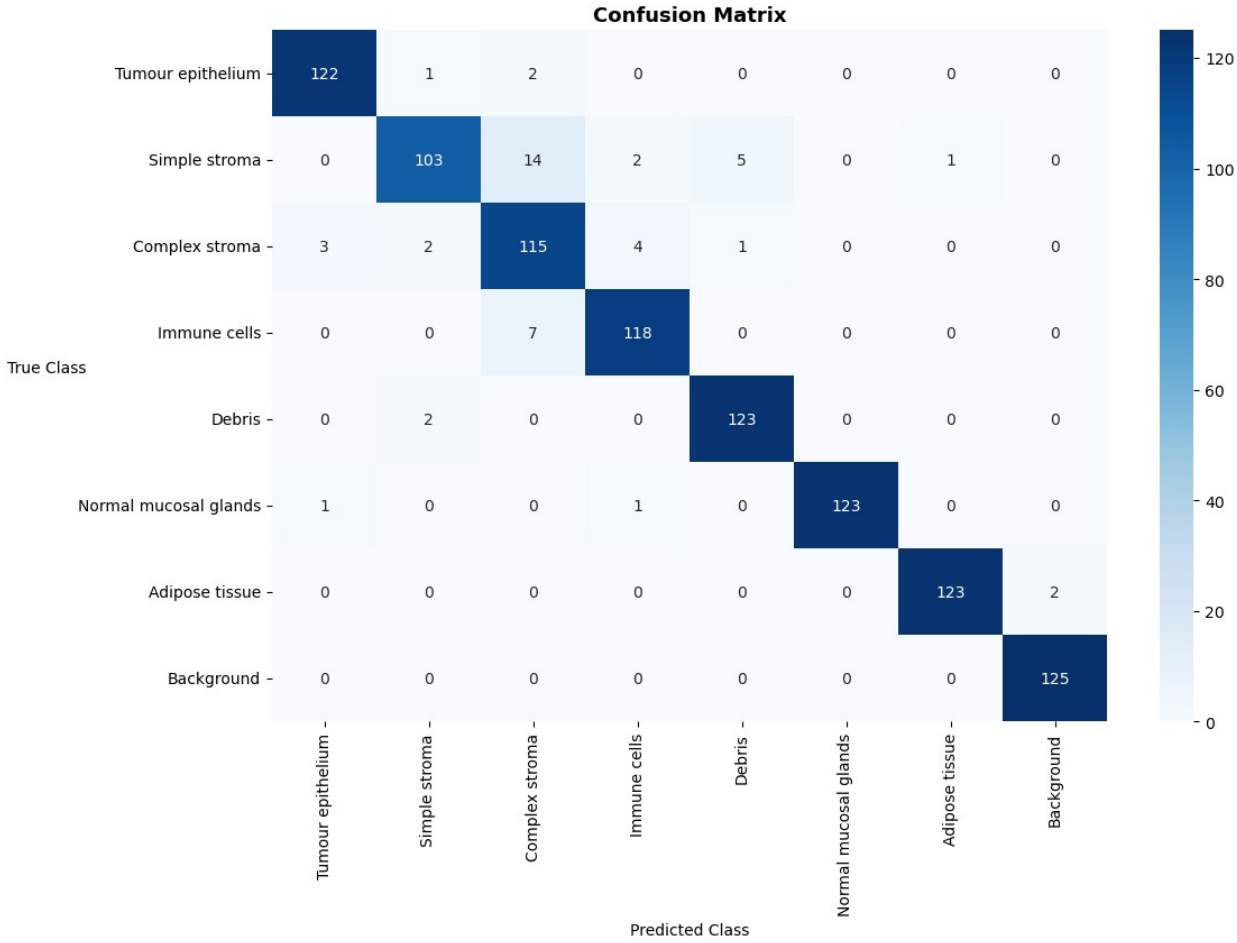
```
0.2670 - val_accuracy: 0.9410 - val_loss: 0.1875 - learning_rate:
1.0000e-05
Epoch 42/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:45 4s/step - accuracy: 0.9058 - loss:
0.3065
Epoch 42: val_accuracy improved from 0.94700 to 0.95000, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 279s 2s/step - accuracy: 0.9040 - loss:
0.3040 - val_accuracy: 0.9500 - val_loss: 0.1622 - learning_rate:
1.0000e-05
Epoch 43/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:38 4s/step - accuracy: 0.9025 - loss:
0.2832
Epoch 43: val_accuracy did not improve from 0.95000
125/125 ━━━━━━━━━━━━━━━━━━━━ 269s 2s/step - accuracy: 0.9062 - loss:
0.2810 - val_accuracy: 0.9360 - val_loss: 0.1882 - learning_rate:
1.0000e-05
Epoch 44/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:36 3s/step - accuracy: 0.9063 - loss:
0.2933
Epoch 44: val_accuracy did not improve from 0.95000
125/125 ━━━━━━━━━━━━━━━━━━━━ 268s 2s/step - accuracy: 0.9005 - loss:
0.2974 - val_accuracy: 0.9470 - val_loss: 0.1751 - learning_rate:
1.0000e-05
Epoch 45/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:36 3s/step - accuracy: 0.9125 - loss:
0.2643
Epoch 45: val_accuracy did not improve from 0.95000
125/125 ━━━━━━━━━━━━━━━━━━━━ 268s 2s/step - accuracy: 0.9080 - loss:
0.2845 - val_accuracy: 0.9450 - val_loss: 0.1792 - learning_rate:
1.0000e-05
Epoch 46/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:35 3s/step - accuracy: 0.9173 - loss:
0.2694
Epoch 46: val_accuracy did not improve from 0.95000
125/125 ━━━━━━━━━━━━━━━━━━━━ 268s 2s/step - accuracy: 0.9118 - loss:
0.2851 - val_accuracy: 0.9370 - val_loss: 0.1820 - learning_rate:
1.0000e-05
Epoch 47/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:39 4s/step - accuracy: 0.9043 - loss:
0.2876
Epoch 47: ReduceLROnPlateau reducing learning rate to
1.9999999494757505e-06.

Epoch 47: val_accuracy did not improve from 0.95000
125/125 ━━━━━━━━━━━━━━━━━━━━ 272s 2s/step - accuracy: 0.9018 - loss:
0.2811 - val_accuracy: 0.9450 - val_loss: 0.1707 - learning_rate:
1.0000e-05
Epoch 48/50
```

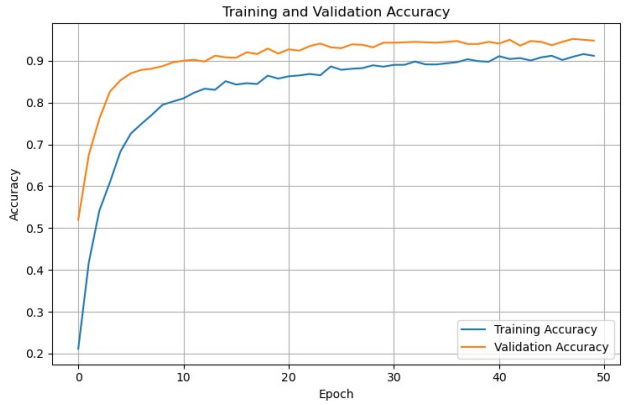
```
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:39 4s/step - accuracy: 0.9128 - loss:
0.2668
Epoch 48: val_accuracy improved from 0.95000 to 0.95200, saving model
to best_model_exp10_224.keras
125/125 ━━━━━━━━━━━━━━━━━━━━ 272s 2s/step - accuracy: 0.9093 - loss:
0.2778 - val_accuracy: 0.9520 - val_loss: 0.1532 - learning_rate:
2.0000e-06
Epoch 49/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:38 4s/step - accuracy: 0.9108 - loss:
0.2757
Epoch 49: val_accuracy did not improve from 0.95200
125/125 ━━━━━━━━━━━━━━━━━━━━ 269s 2s/step - accuracy: 0.9158 - loss:
0.2668 - val_accuracy: 0.9500 - val_loss: 0.1562 - learning_rate:
2.0000e-06
Epoch 50/50
 63/125 ━━━━━━━━━━━━━━━━━━━━ 3:33 3s/step - accuracy: 0.9155 - loss:
0.2678
Epoch 50: val_accuracy did not improve from 0.95200
125/125 ━━━━━━━━━━━━━━━━━━━━ 266s 2s/step - accuracy: 0.9118 - loss:
0.2718 - val_accuracy: 0.9480 - val_loss: 0.1571 - learning_rate:
2.0000e-06
Restoring model weights from the end of the best epoch: 48.
```

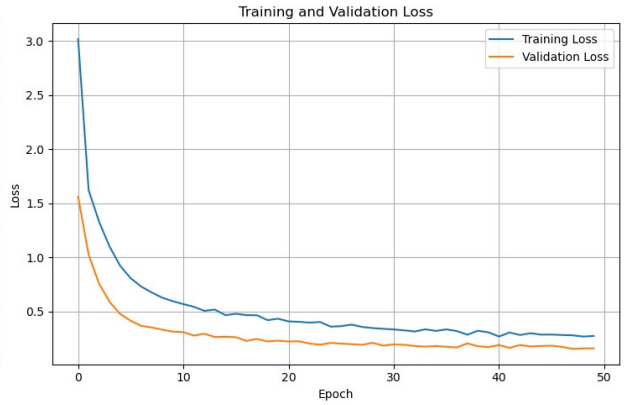
```python
# Setting the model for evaluation.
best_model_224 = tf.keras.models.load_model(checkpoint_filepath)

# Begining the evalutation proccess.
print("10th Experiment Model evaluation - 2nd Attempt")

# Predictions
y_pred_train = np.argmax(best_model_224.predict(x_train_resized),
axis=1) # Get class predictions for training data.
y_pred_test = np.argmax(best_model_224.predict(x_test_resized),
axis=1) # Get class predictions for testing data.

# Evalution Function and Plotting evalutation graphs and confusion
matrix.
results_224 = evaluation(y_pred_train, y_pred_test, y_train, y_test)
plot_vgg_history(history)
```

```
10th Experiment Model evaluation - 2nd Attempt
125/125 ━━━━━━━━━━━━━━━━━━━━ 202s 2s/step
32/32 ━━━━━━━━━━━━━━━━━━━━ 50s 2s/step
Evaluation Report:
Training Accuracy: 0.959
Testing Accuracy: 0.952
Training F1 Score: 0.959
Testing F1 Score: 0.952
Training Precision: 0.960
Testing Precision: 0.954
```

```
Training Recall: 0.959
Testing Recall: 0.952
Overfitting Check: 0.007
Insignificant overfitting.
```








```
# Saving the model to the computer.
best_model_224.save('best_model_10th.keras')
```